1. Design an OTP platform.
2. What can Redis be used for apart from caching?
3. Explain your project.
4. Tell me about yourself.
5. Explain lazy and eager operations in Streams.
6. Write/implement a Circuit Breaker pattern.
7. How would you debug a slow method that contains a database call and a downstream service call?
8. Explain the N+1 problem.
9. Explain "@Transactional" and its attributes.
10. Explain the Saga design pattern.

Round 2

Java / Coding

11. Find the longest substring without duplicate characters.
12. What is treeification in HashMap?
13. What is the output of:

```
Integer a = 128;
Integer b = 128;

System.out.println(a == b);
```

14. What is the value of an Integer?

15. Write a code snippet that causes an OutOfMemoryError.

16. How would you debug an OOM issue?

REST / API

17. What is content negotiation?
18. What does an API return?
19. How do you convert an API response from JSON to XML?
20. What is API versioning?
21. Explain the complete flow and methods of an API for addition.

Security

22. Who creates the JWT?

If there is a fifty APIs, how we will handle the exceptions?

If we have 50 APIs in our Spring Boot application, we don't need to handle exceptions individually in every API. We can use @RestControllerAdvice to implement global exception handling.

We create custom exceptions for specific scenarios and handle them using @ExceptionHandler. This helps us maintain a centralized exception-handling mechanism and return consistent error responses across all APIs.

We can also have a generic exception handler to handle unexpected exceptions

Component vs Bean

Both @Component and @Bean are used to register objects in the Spring Application Context, but they work differently.

1. @Component: It is a class-level annotation. Spring automatically detects the class through component scanning and creates and manages its object (bean).
2. @Bean: It is a method-level annotation. We manually define a method that returns an object, and Spring registers that returned object as a bean. It is generally used inside a class annotated with @Configuration.

Things is working on locally but not in production

"If something is working in my local environment but not working in production, I would debug it step by step.

First, I would understand the exact issue, whether the entire application is failing or only a particular API.

Then I would check production application logs to identify any exceptions or error messages.

Next, I would compare the local and production environments, especially Java versions, application properties, environment variables, database configurations and active profiles.

I would also check database connectivity, external API dependencies, network issues and authentication.

Since we deploy applications using Docker and Kubernetes, I would check whether the correct image is deployed, whether pods are running properly, and whether there are any restarts or resource-related issues.

Finally, after identifying the root cause, I would fix it, test it in a lower environment, deploy the fix and monitor production to ensure the issue is resolved."

Optimistic locking vs pessimistic locking

Imagine two developers updating the same database row at the same time. One of them will have their update rejected. How should the system handle this?

There are two common approaches.

Optimistic locking :assumes conflicts are rare. Both users read the data without acquiring any lock. Each record carries a version number. When a user attempts to write, the database checks: does the version in your update match the current version in the database? If another transaction already incremented the version from 1 to 2, your update still references version 1. The write is rejected.

Pessimistic locking : takes the opposite approach. It assumes conflicts are likely, so it blocks them before they happen. The first transaction locks the row, and every other transaction waits until that lock is released. No version checks needed.

How do you handle security in microservices?

" In microservices, we mainly handle security using Spring Security and JWT-based authentication.

First, when a user logs in, we validate their credentials. If the credentials are correct, we generate a JWT token and send it to the client.

For subsequent requests, the client sends that token in the Authorization header. The API Gateway validates the token and allows only authenticated requests to reach the respective microservices.

We also implement role-based authorization, so users can access only the APIs they're permitted to access.

API rate limiting

Restrict how many requests a client can send to an API within a specific time period.

For example, if a user sends 100 requests within one minute, but our API allows only 50 requests per minute, we reject the remaining requests with HTTP 429 – Too Many Requests.

How do you achieve scalability in your microservices?

"We run multiple instances of each microservice and use load balancing to distribute incoming requests among healthy instances. An API Gateway acts as the entry point and routes requests to the appropriate microservice."

Java Memory Management

Java manages memory automatically through the JVM. The main memory areas are the **Stack**, **Heap**, **Method Area (Metaspace)**, and the **PC Register**. Local variables and object references are stored in the Stack, while objects are allocated in the Heap. Class metadata and static variables are stored in the Method Area. When an object is created, memory is allocated in the Heap, and its reference is stored in the Stack. Once an object is no longer reachable by any live reference, it becomes eligible for garbage collection, and the JVM reclaims its memory automatically. For String literals, Java uses the **String Constant Pool** to reuse identical strings and reduce memory usage. This automatic memory management helps prevent many memory-related issues, although memory leaks can still occur if objects remain referenced unnecessarily.

S - Single Responsibility: A class should have only one responsibility and one reason to change. For example, keep business logic in a service and database operations in a repository.

O - Open/Closed: Classes should be open for extension but closed for modification. For example, add a new payment type by implementing a Payment interface instead of changing existing code.

L - Liskov Substitution: A subclass should be replaceable for its parent without breaking functionality. For example, a Penguin should not extend a class that requires fly().

I - Interface Segregation: Don't force classes to implement methods they don't need. Split large interfaces into smaller, focused ones.

D - Dependency Inversion: Depend on interfaces rather than concrete classes. In Spring Boot, constructor injection of interfaces is a common example.

The **core order of mvn steps**

validate → compile → test → package → verify → install → deploy

7. how to create spring boot application from scratch in technical terms?

To create a Spring Boot application from scratch, I start with Spring Initializr and select Maven, the required Java version, and dependencies such as Spring Web, Spring Data JPA, the database driver, and Lombok. Maven generates the project structure and downloads all dependencies defined in the pom.xml. I then create the main class annotated with @SpringBootApplication, which enables auto-configuration, component scanning, and Java-based configuration. Next, I configure the application in application.properties, create the Entity, Repository, Service, and Controller layers following a layered architecture, and implement the required REST APIs. After testing the application locally using Postman or Swagger, I package it into an executable JAR using mvn clean package. Finally, I create a Docker image, push it to a registry, and deploy it to Kubernetes using Deployment and Service YAML files."

How do you create immutable objects in Java

- You **cannot directly create an immutable object** unless its **class is designed to be immutable**.
- An object's immutability comes from the design of its class.

String isEmpty vs isBlank:
IsEmp:when lenth is 0,
IsBlank: len 0 or spaces in string.

Why is it called CopyOnWrite?

Whenever you perform a write operation (add(), remove(), set()), it creates a **new copy** of the underlying array, applies the modification to the new array, and then replaces the old array.

1. What are the rules for a functional interface?

A **functional interface** is an interface that contains **exactly one abstract method (SAM - Single Abstract Method)**.

Rules

1. It must have **only one abstract method**.
2. It can have **multiple default methods**.
3. It can have **multiple static methods**.
4. Methods inherited from Object (toString(), equals(), hashCode()) do **not** count as abstract methods.
5. It is optional but recommended to annotate it with @FunctionalInterface.

1. Describe step-by-step how to implement JWT Authentication with Spring Security.

Step 1: Add dependencies

Add Spring Security and JWT libraries.

Step 2: Create a User Entity

Store the password in **encoded** form (never plain text).

Step 3: Implement UserDetailsService

Spring Security calls this service to load user details.

Step 4: Configure Password Encoder

Step 5: Create JWT Utility Class

This class

- Generates JWT Token
- Validates Token
- Extracts Username

- Checks Expiration

Step 6: Create JWT Authentication Filter

Create a filter extending

OncePerRequestFilter

Inside doFilterInternal()

- Read Authorization Header
- Check "Bearer "
- Extract JWT
- Validate Token
- Extract Username
- Load UserDetails
- Set Authentication into SecurityContextHolder

Step 7: Configure Security @Bean SecurityFilterChain security(HttpSecurity http) throws Exception { http .csrf(csrf -> csrf.disable()) .authorizeHttpRequests(auth -> auth .requestMatchers("/login").permitAll() .anyRequest().authenticated()); return http.build(); }

Deployment steps

1. Developer pushes code to Git.
2. Jenkins pipeline is triggered automatically.
3. Maven compiles the code and runs tests.
4. SonarQube performs static code analysis.
5. A Spring Boot JAR is generated.
6. Docker builds a container image.
7. The image is pushed to a registry.
8. Helm upgrades or installs the application on Kubernetes.
9. Kubernetes performs a rolling update with minimal downtime.
10. Health checks verify the application before routing traffic.

Marker Interface in Java

A Marker Interface in Java is an interface that contains no methods or fields. It is used to mark a class so that the Java runtime or compiler can identify some special behavior or capability of that class.

public interface Serializable { }

Here, Serializable has no methods but if a class implements it, Java knows that objects of that class can be serialized.

Role of Marker Interfaces

Marker interfaces give information to the JVM or compiler about:

- How to treat the object
- What operation is allowed or disallowed
- What special behavior should be applied

Diamond problem:

The Diamond Problem occurs in multiple inheritance when a class inherits from two parent classes that have a common superclass. If both parent classes provide the same method, the child class cannot determine which implementation to use, causing ambiguity. Java avoids this problem by not allowing multiple inheritance with classes. However, Java supports multiple inheritance through interfaces, and if two interfaces have the same default method, the implementing class must override that method to resolve the conflict.

Lymbda expression:

Its a way to implement functional interface. It provied the implemntation.

Collection vs collections

Collection is a interface,it's the root interface for collection types .

Collections: its a utility class help in sorting,searching reversing.

1 How APIs Are Designed End-to-End

API design starts with understanding business requirements and identifying resources. I design REST endpoints using appropriate HTTP methods, define request and response JSON formats, add validation, security, and error handling, document the APIs using Swagger, implement them using Controller-Service-Repository architecture in Spring Boot, test them, and deploy them. For future changes, I use API versioning to maintain backward compatibility.

🔹 2.Why is String immutable in Java?

There are several important reasons:

1. **String Pool**
    a. Multiple variables can safely share the same string object.
    b. Saves memory.
2. **Security**
    a. Strings are used for usernames, passwords, file paths, URLs, etc.
    b. If strings were mutable, these values could be changed unexpectedly.
3. **Thread Safety**
    a. Immutable objects are naturally thread-safe.
    b. Multiple threads can use the same string without synchronization.

.

🔹 3.Internal working of HashMap in Java 8.

HashMap stores data in the form of key-value pairs. Internally, it uses an array of buckets where each bucket can contain a linked list or a red-black tree.

Hashmap stores the data in key value paires. After we use use map.put() first step would be to get the hashcode from the key using key.hascodde() next step to get the index usinge (n-1)* hash, this index will determine in which buucket it will get stored, if the bucket is empty insert node , if existing the collision happens

Before Java 8, collisions were handled using a **LinkedList**.

From **Java 8 onward**, if:

- Bucket size > 8
- Table size ≥ 64

the linked list is converted into a **Red-Black Tree** for faster lookups.

This improves search time from **O(n)** to **O(log n)**.

Threshold: number of entries a hashmap can hold before it resizes;

Deafult load factor=0.75, capacity:16

Threshold=capa*load

Threshold = 16 × 0.75 = 12

This means when you insert the **13th element**, HashMap will resize its internal array from 16 to 32 buckets.

Why do we need a threshold?

If too many entries are stored in a small number of buckets:

- More collisions occur

What is Hashing?

Hashing is the process of converting a key into an integer value called a **hash code**.

Example:

String key = "Java";

int hash = key.hashCode();

That hash code helps determine **where the key-value pair should be stored**.

4.Why are both hashCode() and equals() used?

hashCode() helps locate the bucket quickly, and equals() confirms whether the keys are actually equal within that bucket. Two different objects can have the same hash code, so equals() is needed to avoid incorrect matches.

🔹 5. Difference between HashMap and ConcurrentHashMap.

Concurrent is thread safe and no null key and values area allowed.

- 7.Can we override static methods?in

No static method are overriden bacuse overriding needed run time poly, but static method resolved at compile time, if a parent class have the same method name it would called method hiding method are called depend on reffernece type not object type.

- 8.Difference between Abstract Class and Interface

Feature	Abstract Class	Interface
Keyword	abstract class	interface
Methods	Can have abstract and concrete methods	Can have abstract, default, and static methods
Variables	Can have instance, static, and final variables	Variables are implicitly public static final
Constructor	Yes	No
Multiple Inheritance	Not supported	Supported
Access Modifiers	Can use any access modifier	Methods are public by default
State (Instance Variables)	Can maintain state	Cannot maintain instance state
Inheritance	extends	implements

- 9.What is the purpose of volatile?

volatile ensures that changes made to a variable by one thread are immediately visible to other threads.

- 10.Explain Synchronization with an example.

->it controlls to accces in multithreaded envirenment, it ensure a single thread can access a critical resourse at at a time so it help to acheve data consistency.

- 11.What causes ConcurrentModificationException?

->fail fast iterator, it occurs when a collection is getting modified while getting itterated except through the iterators own method.

System.out.println("Start small. Ship something.");

```
List<Integer> list=new ArrayList<>();
```

```
        list.add(7);
        list.add(8);
        Iterator<Integer> it=list.iterator();
        while(it.hasNext()){
          Integer value=it.next();
          if(value==8){
            list.remove(value);
          }
        }
        System.out.println(list);
    }
}
```

How many elements can ConcurrentHashMap process at a time? What about HashMap?

HashMap is not thread-safe and does not support concurrent modifications safely. ConcurrentHashMap is thread-safe. Before Java 8, it used segment locking with a default concurrency level of 16, allowing up to 16 threads to write concurrently. From Java 8 onwards, it uses CAS and bucket-level synchronization instead of segments, so there is no fixed limit like 16; multiple threads can read and write concurrently depending on bucket distribution and system resources.

- 12.Difference between fail-fast and fail-safe iterators.

Difference Between Fail-Fast and Fail-Safe Iterators

Feature	Fail-Fast Iterator	Fail-Safe Iterator
Behavior on modification during iteration	Throws `ConcurrentModificationException`	Does not throw exception
Iterates over	Original collection	A copy/snapshot of the collection
Reflects latest changes	No	No (works on snapshot)
Performance	Faster, less memory overhead	More memory overhead due to copying
Thread Safety	Not thread-safe	Safer for concurrent modifications
Examples	`ArrayList`, `HashMap`, `HashSet`	CopyOnWriteArrayList, `ConcurrentHashMap` iterators

🔹 13.Difference between map() and flatMap().

flatMap() flattens multiple streams into a single stream. It unpacked the nested list

Map: it transform the each elemnt 1 to 1

✅ map() – Transforms one element into one element (1 → 1)
✅ flatMap() – Transforms one element into multiple elements and flattens the result into a single stream.

💡 The easiest way to remember it:

map() → One input ➜ One output
flatMap() → One input ➜ Many outputs ➜ Flatten into one stream

🔹 17.What happens internally when Spring Boot starts?

When Spring Boot starts, the JVM executes the main method and calls SpringApplication.run(). Spring Boot creates the ApplicationContext, loads configuration from application.properties or YAML files, performs component scanning, applies auto-configuration based on dependencies, creates and injects beans, starts the embedded

Tomcat server, registers the DispatcherServlet, and finally publishes the ApplicationReadyEvent. After that, the application is ready to accept client requests.

The **DispatcherServlet** is the **Front Controller** of the Spring MVC framework.

It receives **every incoming HTTP request**, decides which controller should handle it, invokes that controller, and returns the response to the client.

18. How does Dependency Injection work?

=using @autowired means asking obj from spring.

19. Why constructor injection is preferred?

= Easier unit testing,Supports immutable fields (final),Detects circular dependency early

20. Difference between @Component, @Service and @Repository.

= @Component, Generic Spring bean

@Service, Business logic layer

@Repository, DAO layer, adds DB exception translation

This is the **main difference** between @Component and @Repository.

Suppose PostgreSQL throws: SQLException

Spring catches it and converts it into a **DataAccessException**, which is an unchecked exception.

@Controller, Handles web requests

21. How does @Transactional work internally?

Internal Working of @Transactional in Spring Boot:

1️⃣ Client calls a service method annotated with @Transactional.

2️⃣ Spring creates a proxy around that method using AOP.

3️⃣ Before executing the method, Spring:

Opens a database connection.
Disables auto-commit mode.
Starts a transaction.

4️⃣ Business logic executes:

```
@Transactional
public void transferMoney(Long from, Long to, Double amount) {
 debit(from, amount);
 credit(to, amount);
```

}

5️⃣ If everything executes successfully:
Spring calls commit().
Changes are permanently saved.
6️⃣ If an exception occurs:
Spring calls rollback().
All changes are reverted.

Finally:

- The connection is released back to the connection pool.

If **@Transactional is not working** Here's a checklist.

1. Method should not get called from the same class

✅ Fix:

- Call the method from another Spring bean.

2. Class is not managed by Spring

❌

UserService service = new UserService();
service.save();

Since you created the object with new, Spring cannot apply the transaction.

@Autowired
private UserService service;

3. Method is private

🔄 4 **Object States in Hibernate**

1. Transient State

A new object created using new, not yet connected to Hibernate.
👉 Example:

Employee emp = new Employee();

emp.setName("John");

✓ Not saved in DB
✓ Not tracked by Hibernate

2. Persistent (Managed) State

The object is now associated with a Hibernate Session.
👉 Example:

session.persist(emp);
emp.setName("John Smith");

✓ Automatically tracked (Dirty Checking)
✓ Changes are synced to DB on commit

3. Detached State

The object was once managed, but session is closed.
👉 Example:

session.close();
emp.setName("Alice");

❌ Changes are NOT tracked

To reattach:
session.merge(emp);

4. Removed State

Entity is marked for deletion.
👉 Example:

session.remove(emp);

✓ Deleted on transaction commit

Transient → Persistent → Detached → Persistent → Removed

💡 **Key Takeaways**

✓ Hibernate tracks only persistent objects
✓ No need for manual update() in managed state
✓ merge() is used for detached objects
✓ Understanding states avoids most persistence bugs

1. How does Spring Boot auto-configuration work internally?

Spring Boot auto-configuration works through @EnableAutoConfiguration. During startup, Spring Boot checks the dependencies available in the classpath and loads matching auto-configuration classes. As a result, components such as Tomcat, DataSource, EntityManager, and DispatcherServlet are configured automatically, reducing manual configuration.

6. Difference between BeanFactory and ApplicationContext?

BeanFactory, ApplicationContext

Basic container, Advanced container

Lazy initialization, Eager singleton creation

No AOP/Event support, Supports AOP/events

Lightweight, Feature rich

ApplicationContext is mostly used.

8. What is propagation in transactions?

Propagation defines **how a transaction behaves when one transactional method calls another transactional method**.

Common types:

REQUIRED → Join existing or create new

REQUIRES_NEW → Always new transaction

MANDATORY → Must have existing transaction

SUPPORTS → Uses existing if available

Example:

@Transactional(propagation = Propagation.REQUIRES_NEW)

9. Difference between Lazy Loading and Eager Loading?

Lazy: Loads when needed, Better performance initially, Default for collections

Eager: Loads immediately,Can cause unnecessary data loading, Default for many-to-one

Example:

@OneToMany(fetch = FetchType.LAZY)

10. What is the N+1 problem in Hibernate? How did you solve it?

Problem:

One query for parent Multiple queries for child records

It causes the performace issue in jpa . It cause larger queries which can slow dwn the apllication

When a table have a relationship with another table that time this n+1 occurs

Example:
1 query for employees + N queries for departments.

Solutions:

JOIN FETCH

@EntityGraph(attributePath=”address”)

This means when you are fetching student details fetch the address also a single quesry not deifferent quesry

Batch fetching

Example:

@Query("SELECT e FROM Employee e JOIN FETCH e.department")

The N+1 problem occurs in Hibernate when we fetch parent records using one query and then Hibernate executes additional queries to fetch related child entities.

For example, if we fetch 10 employees and access their departments, Hibernate might execute one query to fetch employees and 10 additional queries to fetch departments.

This increases database calls and affects application performance.

To solve this, we can use JOIN FETCH, @EntityGraph or batch fetching.

In JOIN FETCH, we fetch the parent and related entities together in a single query. Using @EntityGraph, we can specify which relationships should be fetched. Batch fetching reduces the number of queries by fetching related entities in groups.

11. Difference between CrudRepository and JpaRepository?

CrudRepository, JpaRepository

Basic CRUD, CRUD + JPA features

save(), delete(), pagination, batch, flush

Minimal features, Advanced features

JpaRepository extends PagingAndSortingRepository.

12. How does Spring Data JPA create queries automatically?

Spring parses method names.

Example:

findByNameAndStatus(String name, String status)

Generated query:

SELECT * FROM employee
WHERE name=? AND status=?

Uses proxy + method name parser internally.

13. Difference between @RestController and @Controller?

@Controller, @RestController

Returns view, Returns JSON/XML

Used with JSP/Thymeleaf, Used for REST APIs

@RestController = @Controller + @ResponseBody

14. How does @RequestBody work internally?

Flow:

HTTP request body received

HttpMessageConverter converts JSON → Java object

Jackson library used internally

Example:

```
@PostMapping
public void save(@RequestBody User user)
```

15. Difference between @PathVariable and @RequestParam?

@PathVariable,

Reads a value from the URL path., Used for Identifying a specific resource.

@RequestParam :Reads a value from the query parameter. Filtering, searching, sorting, pagination, optional inputs.

16. How do you handle global exception handling using @ControllerAdvice?

Example:

```
@ControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(Exception.class)
  public ResponseEntity<String> handle(Exception ex){
    return ResponseEntity.badRequest().body(ex.getMessage());
  }
}
```

Benefits:

Centralized exception handling

Cleaner controllers

17. Difference between Filters and Interceptors?

Interceptor

An **Interceptor** is a **Spring MVC** feature.

It works **after the request enters Spring MVC** but **before the controller**.

Filter

1. Filter → HTTP/request level

A Filter works very early, at the **Servlet/HTTP level.**

For example, you want to add a header to every response:

A **Filter** is part of the **Servlet API**.

- Executes **before the request reaches Spring**.
- Can modify both the request and response.
- Configured for URLs.

18. How does Spring Security filter chain work?

Spring Security uses multiple filters.

Flow:
Request → Security Filter Chain → Authentication → Authorization → Controller

Spring Security uses a filter-based architecture. Every request first passes through DelegatingFilterProxy, which forwards to FilterChainProxy. FilterChainProxy executes a chain of security filters such as authentication, JWT validation, authorization, and exception handling filters. If authentication and authorization succeed, the request reaches the controller; otherwise, Spring Security returns a 401 or 403 response. The authenticated user's details are stored in SecurityContextHolder and are available throughout the request lifecycle.

In my applications, I handle security using Spring Security with JWT-based authentication. The general flow is:

1. The user logs in with a username and password.
2. The credentials are authenticated using Spring Security's AuthenticationManager.
3. If authentication succeeds, the application generates a JWT token.
4. The client includes this JWT in the Authorization header (Bearer <token>) on every subsequent request.
5. A JWT filter validates the token on each request. If it's valid, the user's authentication is stored in the Security Context, and the request proceeds to the controller.

19. How JWT authentication works internally?

JWT (JSON Web Token) is one of the most widely used authentication mechanisms in modern web applications. Here's how it works:

✅ 1. User Login
The user submits credentials (email & password).

✅ 2. Credential Validation
The server verifies the credentials against the database.

✅ 3. JWT Generation
If authentication succeeds, the server generates a JWT containing:

* Header
* Payload (user information/claims)
* Signature

✅ 4. Token Storage
The client stores the token securely (preferably in HttpOnly Cookies).

✅ 5. Authenticated Requests
The token is sent with every request using the Authorization header:

✅ 6. Token Verification
The server validates:

* Token signature
* Expiration time
* User claims/permissions

✅ 7. Access Protected Resources
If the token is valid, the user gains access to secured APIs and resources.

💡 Why JWT?
✓ Stateless Authentication
✓ Scalable for Microservices
✓ Reduced Server-Side Session Management
✓ Works seamlessly with REST APIs

The user logs in using a username and password. Spring Security validates those credentials against the database (typically through UserDetailsService and PasswordEncoder). If the credentials are valid, the server generates a JWT token and sends it back to the client. The client stores this token and includes it in the Authorization header (Bearer <token>) for every subsequent API request. For each request, a JWT filter intercepts the request, extracts and validates the token, and if the token is valid, Spring Security creates an authentication object and stores it in the SecurityContextHolder. The request is then allowed to access the protected API.



JWT contains:

Header

Payload

Signature

20. Difference between authentication and authorization?

Authentication, Authorization

Who are you?, What can you access?

Login, Permissions

Example:

Authentication → username/password

Authorization → ADMIN role access

Why does Map not extend Collection?

Because Map works differently. Collection stores single values, but Map stores key-value pairs. Their structure and methods are different, so Map is kept separate.

Explain the internal working of HashMap?

A HashMap stores data in the form of key-value pairs and uses a technique called hashing to store and find data quickly.

Lets understand step by step

1) Hashing of key: When you add data, Java takes the key and calls its hashCode() to generate a number.

2) Bucket location: This number is used to decide which bucket (index) in an internal array the data should go to.

3) Storing the entry: The key and value are stored together as an object (called Entry/Node) in that bucket.

4) Handling collisions

If two keys give the same bucket: They are stored in a linked list (older Java). If many entries collide, it becomes a tree (Java 8+) for faster search

5) Retrieving data

When you call get(key):

- Java finds the bucket using hashCode()
- Then uses equals() to find the exact key
- Returns the value

GenZ Career on Youtube

What if equals() is true but hashCode() is different?

HashMap will treat them as different keys and store both. Later, searching may fail because it looks in a different bucket. This breaks HashMap be



22. RestTemplate vs WebClient vs Feign Client?

1. RestTemplate

What is it?

- A synchronous (blocking) HTTP client.
- When Service A calls Service B, the thread waits until the response is received.

2. Feign Client

What is it?

- A declarative REST client.
- You only define an interface, and Spring generates the implementation.

3. WebClient

What is it?

- Modern HTTP client introduced with Spring WebFlux.
- Supports **non-blocking** communication.

A **cascading failure** in microservices happens when the failure of one service causes other dependent services to fail, spreading the issue across the system.

23. How did you implement Circuit Breaker in microservices?

Used Resilience4j.

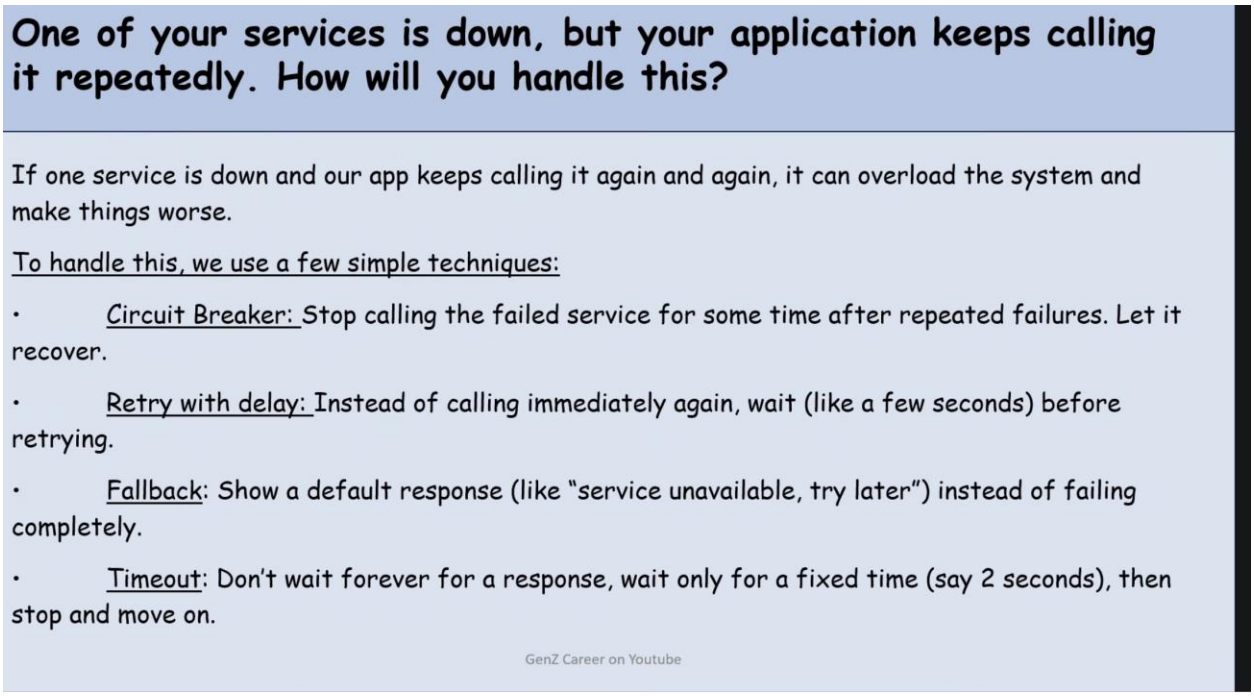
<artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>

@CircuitBreaker(name="paymentService", fallbackMethod="fallback")

```
resilience4j:
  circuitbreaker:
    instances:
      inventoryService:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
```

This means:

- Keep the last **10 requests**.
- If **50% fail**, open the circuit.
- Wait **10 seconds** before trying again.



24. How do you handle distributed transactions in microservices?

Usually using:

Saga Pattern

Event-driven approach

Avoid 2PC because expensive.

Types:

1. Choreography

In choreography, **each microservice knows what to do next**.

There is **no central coordinator**.

Services communicate using **events** (often via Kafka).

Orchestration

In orchestration, a **central orchestrator** controls the workflow.

Services do not decide what happens next.

Compensation transactions used for rollback.

Centralized is a more general concept where a single component provides a shared capability, such as centralized configuration (Spring Cloud Config), centralized routing (API Gateway), or centralized authentication. Orchestration is one type of centralized control, but not every centralized component is an orchestrator.

How Saga Pattern automatically undoes, could you please explain its mechanism thoroughly?

Saga Pattern works like a step-by-step transaction with a backup plan.

- A big process is broken into smaller steps (Example, Order → Payment → Inventory).
- Each step completes its own work and saves it.
- If all steps succeed → process is done
- If any step fails → system runs compensating actions (undo steps).

Example: If Payment succeeds but Inventory fails, then Saga triggers a refund to undo the payment.

Could you please explain Choreography and Orchestration styles of saga pattern?

Choreography:

In choreography, there is no central controller. Each service works independently and reacts to events. For example, when Order Service creates an order, it sends an event. Payment Service listens to it, processes payment, and then sends another event. Inventory Service listens and continues the process. So basically, services talk through events and decide themselves what to do next.

Orchestration:

In orchestration, there is a central coordinator (orchestrator) that controls everything. It tells each service what to do step by step. For example, the orchestrator tells Payment Service to process payment, then tells Inventory Service to update stock. If something fails, it also tells services how to undo actions. So here, one central service manages the whole flow.

Each microservice has its own database, but sometimes you need data from another service. So will you call another service every time or duplicate the data?

In microservices, we should not call another service every time, because it can slow things down and create dependency between services.

Instead, a common approach is to duplicate only the required data (called data replication). Services share updates through events, so each service keeps a small copy of the data it needs. This makes systems faster and

28. How do you externalize configuration in Spring Boot?

Methods:

application.properties

Environment variables

Config Server

Kubernetes ConfigMap/Secrets

Example:

server.port=8081

Using:

@Value("${server.port}")

29. How do you troubleshoot slow APIs in production?

When an API becomes slow in production, I first check monitoring dashboards and logs to understand when the issue started and how much the response time has increased. Then I try to identify where the delay is occurring—whether it's in the application code, database, external service calls, or infrastructure. I review slow database queries, check for missing indexes, analyze thread and memory usage, and look for any recent deployments or configuration changes that might have caused the issue. If the API depends on other services, I use tracing and logs to find slow downstream calls. Once I identify the bottleneck, I fix the root cause and monitor the API to ensure performance returns to normal.

If they ask for a real example:

In one case, an API response time increased significantly. After checking the logs, I found that a database query was taking several seconds to execute. We analyzed the query, identified a missing index, and after adding the index the response time dropped from a few seconds to a few hundred milliseconds. This resolved the issue and improved the overall user experience.

30. What production issue did you face in Spring Boot and how did you fix it?

Example Answer:

"We faced high API response time in production. After analysis, we found Hibernate N+1 query issue causing multiple DB calls. We fixed it using JOIN FETCH and pagination. Response time reduced from 8 seconds to below 1 second."

Another good example:

Kafka consumer lag

Memory leak

DB connection exhaustion

Thread pool saturation

Try with resourse:

Exactly! That is the entire purpose of try-with-resources. You **do not** need a finally block to close your resources anymore. Java automatically handles the cleanup behind the scenes.

Can you still use a finally block?

Yes, you can, but only if you need to do something *other* than closing resources. If you want to log a message when the process completes, trigger an alert, or reset a global variable, you can still append catch and finally blocks to a try-with-resources statement.

```
//  Modern, clean way
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
   System.out.println(br.readLine());
} catch (IOException e) {
   e.printStackTrace();
} // Java automatically closes 'br' right here!
```

Iterator vs Enumeration in Java:

Both are used for traverse collection but iteration is newer and more powerfull. .remove() is supported in iterator.to read elemnt hashMoreElement ,nextelem is used and in iterator hashNext(),next() is used.

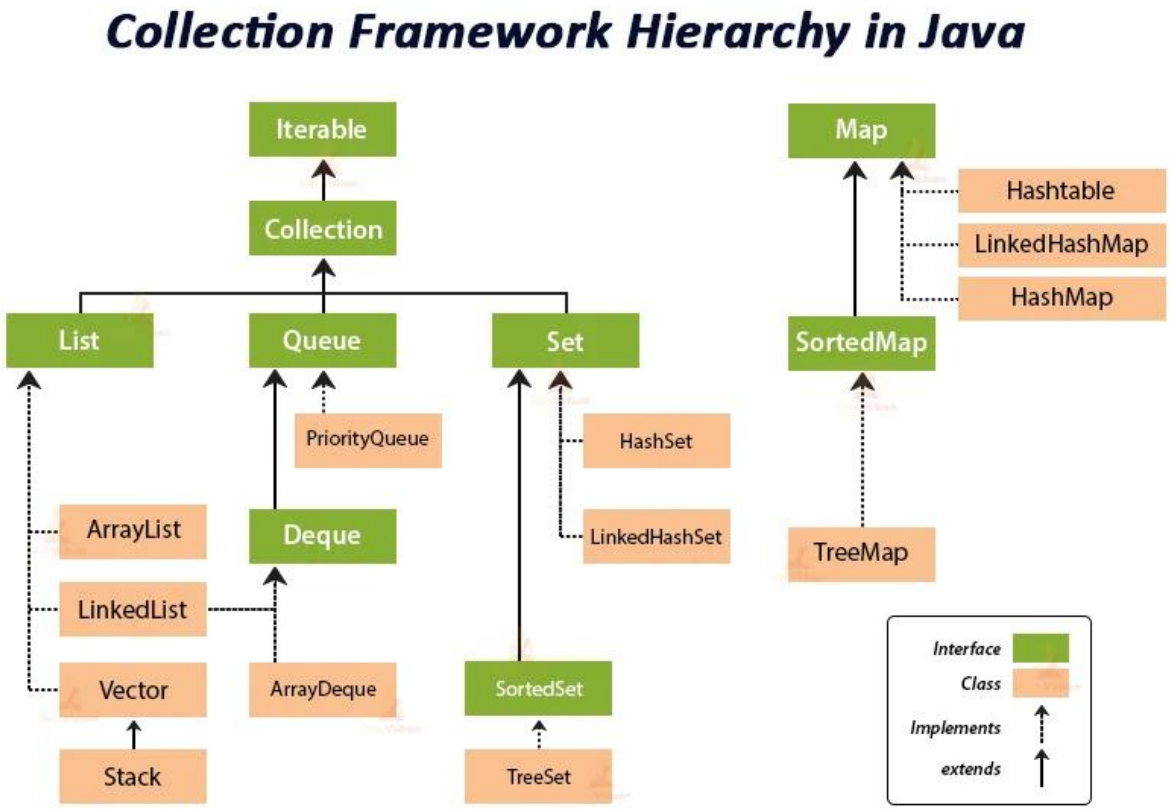
Iterable and iterator

"Iterable is an interface that represents a collection whose elements can be traversed. It provides the iterator() method to create an Iterator. The Iterator is the object that actually moves through the collection using hasNext() and next(). In simple terms, Iterable gives you the iterator, and Iterator does the iteration."

Memory Trick

- **Iterable = Gives an iterator**
- **Iterator = Goes through the elements**

Iterable is an interface that represents a collection of elements that can be iterated over. It contains a single method, iterator(), which returns an Iterator. Any class implementing Iterable can be used in the enhanced for-each loop. **Iterator** is used to traverse the collection and provides methods like hasNext(), next(), and remove(). In short, Iterable provides the iterator, while Iterator performs the iteration.



Types of Functional Interfaces in Java 8

```
List<String> names = Arrays.asList("Java", "Spring", "Boot");

names.stream()
    .filter(name -> name.length() > 4) // Predicate
```

```
    .map(String::toUpperCase)        // Function
    .forEach(System.out::println);    // Consumer
```

Interview Answer

The four main built-in functional interfaces in Java 8 are:

- **Predicate<T>**: Takes an input and returns a boolean.
- **Function<T,R>**: Takes an input and returns a result.
- **Consumer<T>**: Takes an input and performs an operation without returning anything.
- **Supplier<T>**: Takes no input and returns a value.

Sealed classes

It give you a third option: Controlled Inheritance. You can say: *"I am creating a GamePiece, and the **only** pieces allowed to exist in this universe are King, Queen, and Knight."*

How it looks in code

You use the sealed keyword to lock the class down, and permits to list your allowed options:

Java

```
// You lock the door and only let these three in
public sealed class GamePiece permits King, Queen, Knight {}
```

If we use sealed class and in code we use switch no neeed defalut one.

switch (piece) { case King k -> System.out.println("Move 1 step."); thus arrow operator remove the need of break after case.

What do you meant by Object-Oriented Programming and Its Principles?

OOP is pragramming which based on objects . It contains data and behavior. It help in resuable maiintanable applcation. It has four principles

- **Encapsulation** – Wrapping data and methods together and controlling access using access modifiers.
- **Inheritance** – Allowing one class to inherit properties and methods from another class for code reuse.
- **Polymorphism** – Allowing the same method to behave differently through overloading and overriding.

- **Abstraction** – Hiding implementation details and exposing only essential functionality through interfaces or abstract classes.

2. What will be the output?
- put("abc","tan");
- put("abc","cho);
- get("abc");

Output=cho

But if we do this strinng value= put("abc","cho)

It will return old value
3. Find the missing number from:
- int[] arr = {1, 2, 5, 6, 3, 7};
- What if the collection is very large? Explain the logic without iterating through all elements.
4. Explain:
- @Autowired is used for **Dependency Injection** in Spring. It tells Spring to automatically inject the required bean.

@PathVariable is used to extract values from the URL path.
- @RequestParam is used to read query parameters from the URL.

6. Difference between Encapsulation and Abstraction.

Real-Life Example

Encapsulation: ATM machine protects your bank account data. You cannot directly access or modify the account balance.

Abstraction: You press the "Withdraw" button on an ATM. You don't need to know how the machine communicates with the bank internally.

Interview Answer (Short)

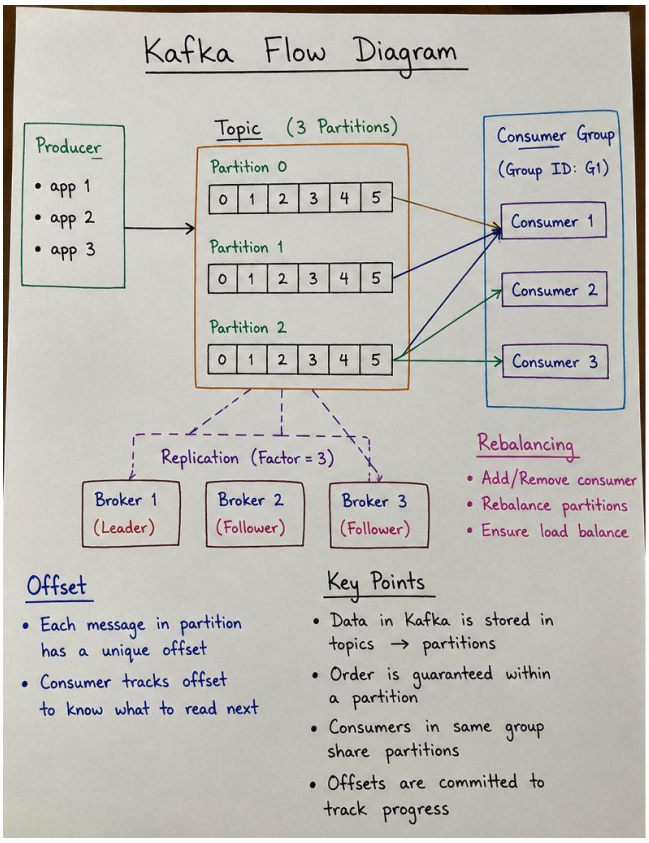
Encapsulation is the process of binding data and methods together and restricting direct access to data using access modifiers. Abstraction is the process of hiding implementation details and exposing only the required functionality to the user. Encapsulation is achieved using private fields and getters/setters, while abstraction is achieved using abstract classes and interfaces.

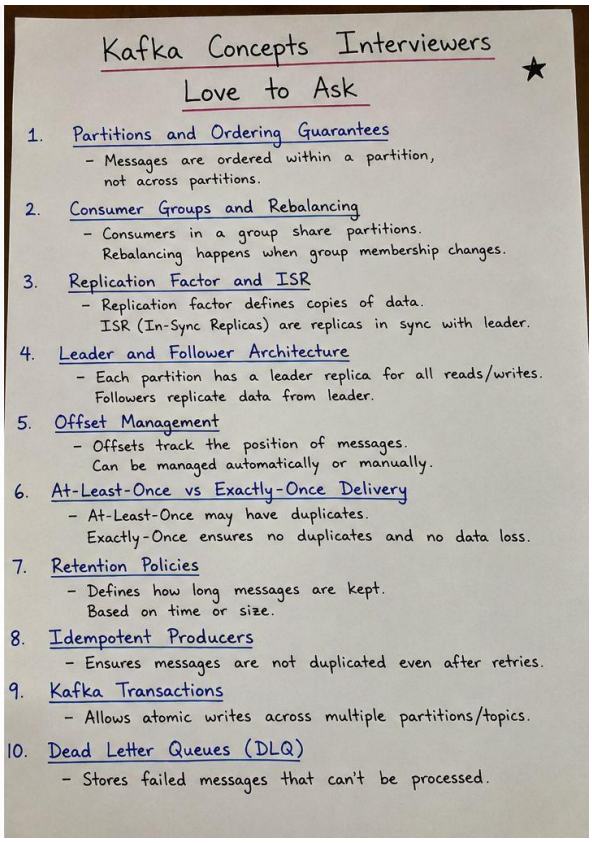
7. How do you create a Custom Exception in Java?

A custom exception is created by extending Exception for checked exceptions or RuntimeException for unchecked exceptions. We usually add constructors that pass error messages to the parent class using super(message). Custom exceptions help represent business-specific error scenarios such as UserNotFoundException, InvalidAgeException, or InsufficientBalanceException.

8. How do you create a Custom Class in Java?

9. Difference between HashMap, Hashtable, and ConcurrentHashMap.





10. How many elements can ConcurrentHashMap process at a time? What about HashMap?

HashMap is not thread-safe. If multiple threads modify it concurrently, it can lead to race conditions and data corruption.

ConcurrentHashMap is thread-safe and allows multiple threads to perform read and write operations simultaneously. In Java 8+, it uses bucket-level locking and CAS instead of locking the entire map, so multiple threads can update different buckets concurrently. There is no fixed limit on the number of threads or elements it can process at once; the level of concurrency depends on how the data is distributed across buckets and the available CPU resources.

11. Difference between PUT and PATCH.

Put used to replace the existing resouce , patch is use for replacing paritial

12. Difference between DROP and TRUNCATE.

Truncate will drop the rows

Drop will remove the table

15. What is Serialization and why is it used?

Serialization = Object → Byte Stream

- Saved to a file
- Sent over a network
- Stored in a database or cache

Deserialization = Byte Stream → Object

Why is transient used?

A: To exclude sensitive or unnecessary fields from serialization, such as passwords or temporary data.

16. How do you handle Exceptions in Java?

"In Java, I handle exceptions using try-catch-finally, throw, and throws. Checked exceptions are handled at compile time, while unchecked exceptions occur at runtime. I always catch specific exceptions instead of generic ones, use try-with-resources for automatic resource management, and create custom exceptions for business validations when required. In production applications, I log exceptions and return meaningful error messages instead of exposing stack traces."

18. If a production issue occurs, how would you investigate it after checking the logs?

My approach is: check logs → reproduce the issue → identify whether it's in my service or a dependency → verify database, APIs, Kafka, and external services → check metrics like CPU, memory, and response time → review recent deployments or configuration changes → fix the root cause, test it, deploy through CI/CD, and monitor production after deployment."

- Difference between Monolithic and Microservices Architecture.

Monolithic Architecture

A **monolithic application** is built as a **single deployable unit**. All modules (UI, business logic, database access, authentication, etc.) are part of one application and are deployed together.

Microservices Architecture

A **microservices architecture** breaks an application into **small, independent services**, where each service is responsible for one business capability. Each service can be developed, deployed, and scaled independently.

i wouldn't migrate the entire application at once because that would be risky. I'd first understand the business domains and identify modules that are relatively independent and don't have too many dependencies on other parts of the system.

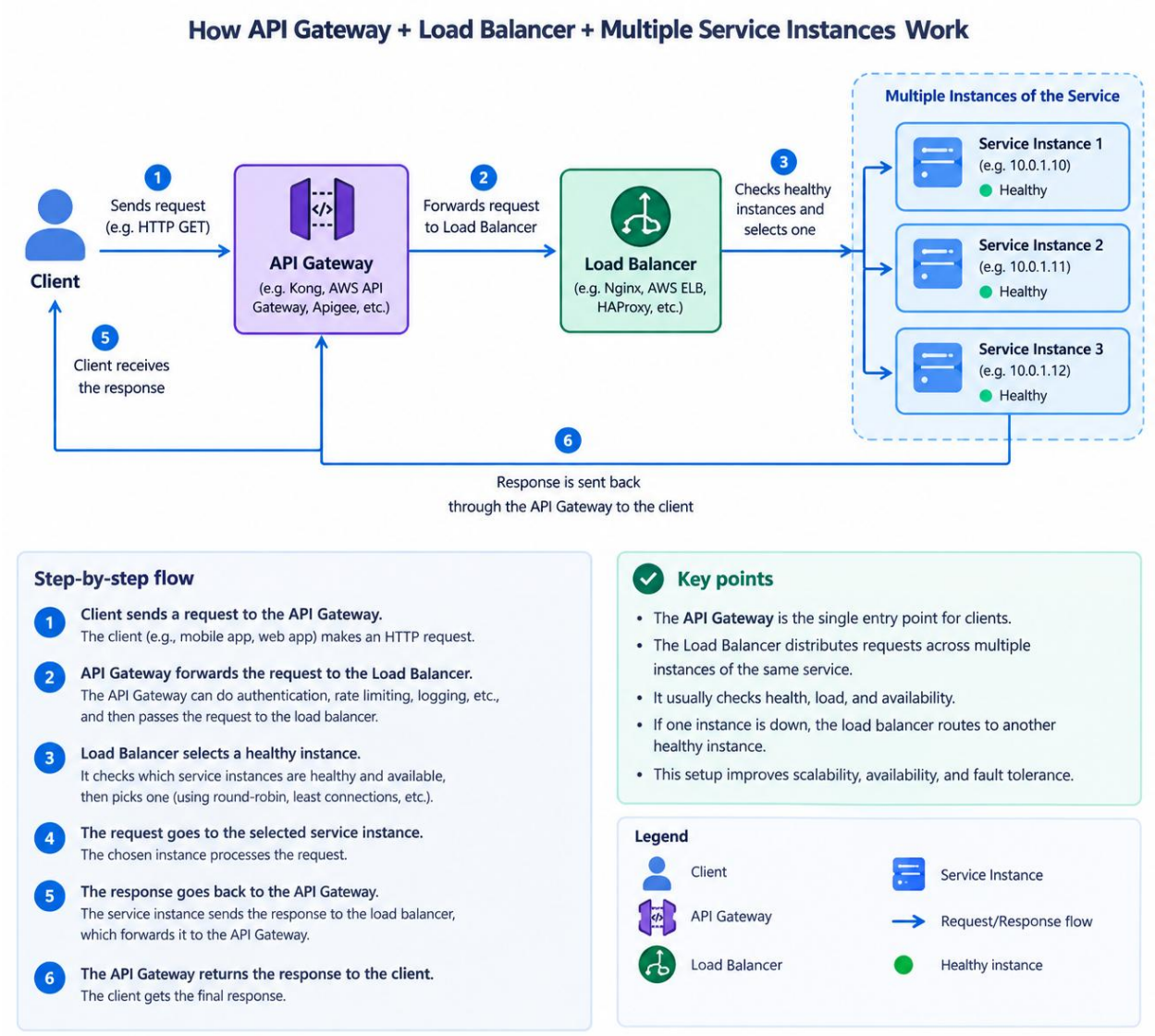
Then I'd look at which modules change frequently or need to scale independently. For example, modules like Notification, Authentication, or Payment are often good candidates because they have well-defined responsibilities.

I'd also avoid starting with modules that are tightly coupled with many others, since they're more complex to separate.

Instead of a big-bang migration, I'd migrate one module at a time, test it thoroughly, and let it communicate with the remaining monolith through APIs. This gradual approach reduces risk and allows us to monitor the system before moving on to the next module.

- What challenges have you faced while working with Microservices?

What is API Gateway and why is it required?

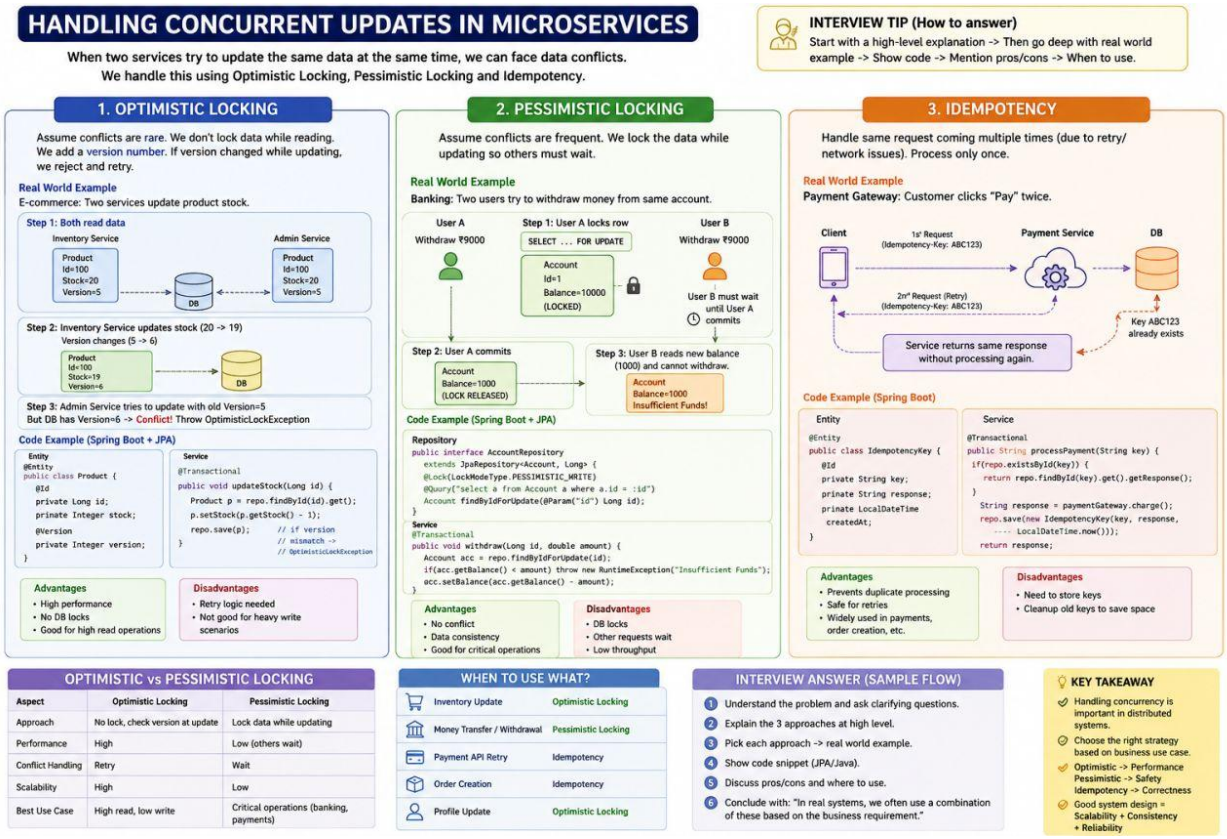


An API Gateway acts as a single entry point for all client requests in a microservices architecture. It routes requests to the appropriate microservices and handles cross-cutting concerns such as authentication, authorization, load balancing, rate limiting, logging, monitoring, and request routing. It simplifies client communication because clients interact with only one endpoint instead of multiple service endpoints. In Spring Boot microservices, Spring Cloud Gateway is commonly used as the API Gateway and often integrates with Eureka for service discovery.

What is Service Discovery? How does Eureka work?

Service Discovery is a mechanism that allows microservices to find and communicate with each other without hardcoding IP addresses or ports. Eureka is Netflix's service registry solution. Each microservice registers itself with the Eureka Server and periodically sends

heartbeats to indicate it is healthy. When one service needs to communicate with another, it queries Eureka using the service name, and Eureka returns the available service instances. This enables dynamic discovery, scalability, and easier management of microservices in distributed systems.



🔹 Difference between ArrayList and LinkedList.

ArrayList is implemented using a dynamic array, while LinkedList is implemented as a doubly linked list. ArrayList provides fast random access with O(1) time complexity because elements can be accessed directly by index. However, inserting or deleting elements in the middle is slower because the remaining elements need to be shifted. LinkedList, on the other hand, doesn't require shifting elements during insertion or deletion, but accessing an element by index is slower because it has to traverse the list. I would choose ArrayList when read operations are more frequent and LinkedList when insertions and deletions are more frequent.

🔹 Difference between Comparable and Comparator.

1. Comparable

Use **Comparable** when your custom class has a **default or natural ordering**.

For example, if employees should normally be sorted by **ID**, implement Comparable.

2. Comparator

Use **Comparator** when you need **different ways of sorting**.

For example:

- By salary
- By name
- By age

"Comparable is used to define the natural or default ordering of objects. The class itself implements the Comparable interface and overrides the compareTo() method. Since a class can have only one natural ordering, Comparable supports only one default sort order.

Comparator is used for custom sorting. It is implemented in a separate class or as a lambda expression and overrides the compare() method. We can create multiple comparators for different sorting requirements, such as sorting employees by salary, name, or age, without modifying the original class."

🔹 What is Optional and why was it introduced?

To avoid null pointer Exception.

- Thus, it solves the problem of *NullPointerPointerException* making the code more readable as compared to null checks.

🔹 Difference between String, StringBuilder, and StringBuffer.

String buffer is thread safe

🔹 Explain ExecutorService and Thread Pool.

ExecutorService is a framework provided by Java to **manage and execute threads efficiently**.

Instead of creating threads manually using new Thread(), we submit tasks to an ExecutorService, which manages a **pool of reusable threads**.

Why do we need it?

Without ExecutorService:

```
new Thread(task1).start();
new Thread(task2).start();
new Thread(task3).start();
```

If your application creates thousands of threads, it can:

- Consume a lot of memory
- Slow down the application
- Increase context switching overhead

With ExecutorService, a fixed number of threads are created once and reused.

What is a Thread Pool?

A **Thread Pool** is a collection of pre-created worker threads.

When a task is submitted:

- If a thread is free, it executes the task.
- If all threads are busy, the task waits in a queue.
- Once a thread finishes, it picks up the next task from the queue.

Execute() vs submit()

execute() and submit() are methods of ExecutorService used to execute tasks. execute() accepts only a Runnable and returns void, making it suitable for fire-and-forget tasks. submit() accepts both Runnable and Callable and returns a Future, which can be used to retrieve the result, wait for completion, or handle exceptions. If I need a return value or task status, I use submit(). Otherwise, I use execute().

Runnable is used when a task doesn't need to return a result. It has a single method, run(), which returns void and cannot throw checked exceptions.

Callable is used when a task needs to return a value or throw checked exceptions. It has the call() method and is typically executed using an ExecutorService, which returns a Future. We use Future to retrieve the result, check task completion, or cancel the task.

In short: Use **Runnable** for fire-and-forget background tasks, and **Callable** when you need a computed result or better exception handling.

🔹 Difference between synchronized and volatile.

volatile guarantees the visibility of changes across threads but does not provide atomicity or mutual exclusion. It is suitable for simple shared flags.

synchronized uses a monitor lock to ensure that only one thread can execute a critical section at a time. It provides both visibility and atomicity for the synchronized block or method but has more overhead due to locking.

🔹 What is a Deadlock? How can you prevent it?

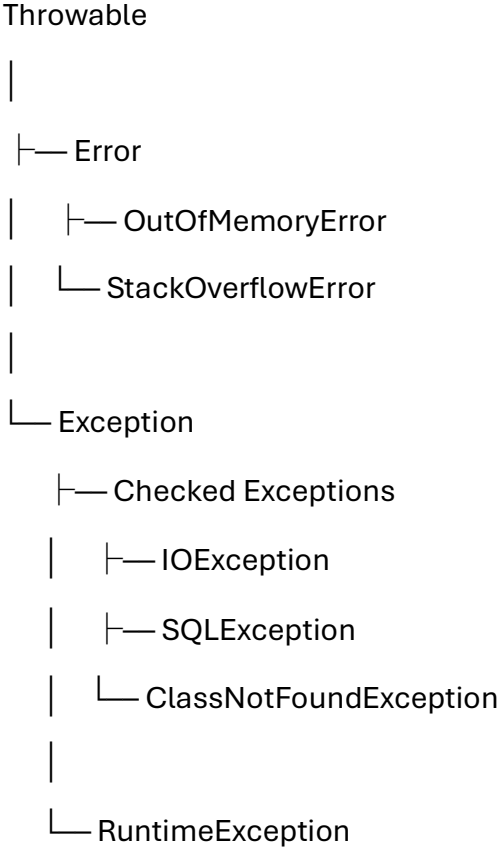
A **deadlock** occurs when **two or more threads are permanently waiting for each other to release resources**, and none of them can proceed.

How does tryLock() help?

It attempts to acquire the lock without blocking forever. If it can't get the lock (or within the specified timeout), it returns false, allowing the thread to retry, back off, or take another action instead of becoming deadlocked.

ReentrantLock with tryLock() when appropriate, so a thread doesn't wait forever for a lock."

🔹 Explain Exception Handling hierarchy in Java.



├── NullPointerException

├── ArithmeticException

├── IllegalArgumentException

├── NumberFormatException

└── ArrayIndexOutOfBoundsException

- Difference between Checked and Unchecked Exceptions.

Difference between @Autowired and Constructor Injection.

@Autowired (Field Injection)

With field injection, Spring injects the dependency directly into the field.

Constructor Injection

The dependency is injected through the constructor.

What is @Transactional and its limitations?

@Transactional is a Spring annotation used to **manage database transactions automatically**.

It ensures that **all database operations inside a method either succeed together or fail together**.

- If everything is successful → **Commit** the transaction.
- If an exception occurs → **Rollback** the transaction.

This follows the **all-or-nothing** principle.

Some limitations are that it only manages a single database transaction, it doesn't work across multiple microservices, self-invocation bypasses transactional proxies, by default it rolls back only for unchecked exceptions, and it's generally applied only to public methods."

Explain JPA Entity Lifecycle States.

🔹 Difference between FetchType.LAZY and FetchType.EAGER.

FetchType.LAZY

Definition:
The related entity is **not loaded immediately**. It is fetched **only when you access it**.

FetchType.EAGER

Definition:
The related entity is **loaded immediately** along with the parent entity.

What causes LazyInitializationException?

LazyInitializationException occurs when we try to access a lazily loaded entity after the Hibernate session has been closed.

Difference between JPQL and Native SQL Query.

JPQL, or Java Persistence Query Language, works with JPA entities and their fields rather than database tables and columns. It is database-independent because Hibernate translates it into the appropriate SQL for the underlying database.

Native SQL, on the other hand, uses the actual database tables and SQL syntax, making it database-specific.

🔹 How do you implement Pagination and Sorting in Spring Data JPA?

Spring Data JPA provides:

- Pageable
- Page
- Sort

These help fetch data page by page instead of loading the entire table.

```
Pageable pageable = PageRequest.of(0, 10);

Page<Employee> page = repository.findAll(pageable);
```

Get the data:

```
List<Employee> employees = page.getContent();
```

Pagination + Sorting Together

```
Pageable pageable =
    PageRequest.of(
        0,
        10,
        Sort.by("salary").descending());

Page<Employee> page =
    repository.findAll(pageable);
```

Spring Data JPA already provides pagination support through JpaRepository.

Explain First-Level and Second-Level Cache.

- What is OAuth2?

OAuth 2.0 is an **authorization framework** that allows a user to grant a third-party application **limited access** to their resources **without sharing their password**.

Start() vs run()

start() creates a new thread, and the JVM internally invokes run() on that new thread. Calling run() directly is just a normal method call and executes on the current thread without creating a new thread.

False sharing

Its cache perfomance isuue in multithreaded application. It occurs When differnet theread updates the different variabke which belong to the same cpu cache line . It can be avoided using padding or @Contended so frequently updated variables occupy separate cache lines.

How to implement a producer-consumer problem?

The **Producer-Consumer Problem** is a classic multithreading problem where:

- **Producer** creates data and puts it into a shared buffer.
- **Consumer** takes data from the shared buffer.
- If the buffer is **full**, the producer waits.
- If the buffer is **empty**, the consumer waits.

producer waits when the buffer is full, and the consumer waits when it's empty. In modern Java, the preferred solution is to use a BlockingQueue, such as ArrayBlockingQueue, because put() and take() handle blocking automatically, making the code simpler, safer, and easier to maintain."

Synchrized: manually no need of lock unlock jvm going to do that

ReentrantLock: manually need to do lock unlock, it provide one more feature try lock

- **lock()** → waits until the lock is free.
- **tryLock()** → checks once and returns true or false immediately.

8. How ConcurrentHashMap works internally

- ✅ **Interview-friendly:** "It uses bucket-level locking."
- ✅ **Technically correct:** "It synchronizes on the **first node** of the bucket."

ConcurrentHashMap in Java 8+ stores entries in an array of buckets, just like HashMap. For a put(), it computes the hash and locates the bucket. If the bucket is empty, it inserts the node using **CAS (Compare-And-Swap)** without locking. If the bucket already contains data, it synchronizes only on that bucket's first node, allowing updates to different buckets to happen concurrently. Reads (get()) are generally lock-free. When collisions become excessive, the linked list is converted into a red-black tree. During resizing, multiple threads can cooperate to transfer buckets, improving scalability under high concurrency. This combination of CAS, fine-grained bucket locking, and cooperative resizing makes ConcurrentHashMap much more scalable than synchronizing an entire HashMap.

CAS (Compare-And-Swap) is a low-level atomic operation used in concurrent programming to update a value **without using locks**. It's one of the key techniques used internally by ConcurrentHashMap, atomic classes like AtomicInteger, and many other concurrent utilities in Java.

How CAS Works

CAS takes **three values**:

1. **Memory location** (the variable to update)
2. **Expected value** (what you think is currently stored)
3. **New value** (what you want to store)

Infosys

Demon thread

A **Daemon Thread** is a **background thread** that provides support to user (non-daemon) threads.

The JVM **does not wait for daemon threads to finish**. Once all user threads have completed, the JVM exits, even if daemon threads are still running.

Real-Life Example

Imagine a restaurant:

- **Customers** → User threads
- **Cleaning staff** → Daemon threads

The cleaning staff works in the background while customers are present. Once all customers leave, the restaurant closes, even if the cleaning staff hasn't finished.

Examples of Daemon Threads

- Garbage Collector (GC)
- Background monitoring threads
- Logging threads
- Cache cleanup threads

```
public static void main(String[] args) {


 MyThread t = new MyThread();


 t.setDaemon(true); // Make it a daemon thread

 t.start();


 System.out.println("Main thread finished");

 }
```

6) Difference between Process and Thread?

A process is an independent program in execution with its own memory space and system resources.

Threads are lightweight units of execution within a process.

In Java, when we run an application, it starts as a single process, and we can create multiple threads using the Thread class or the Runnable interface to perform tasks concurrently.

7) Different ways to create a Thread and implement using Runnable

->Extending Thread class

->Implementing Runnable

->Using ExecutorService

10) What is Thread Scope?

Thread Scope means a bean is created **once per thread**. Every thread gets its own instance of the bean, and that instance is not shared with other threads.

- It is **not enabled by default** in Spring.
- You need to register SimpleThreadScope to use it.

Example

```
@Scope("thread")
@Component
public class UserContext {
}
```

If:

- Thread A requests UserContext → gets Instance A.
- Thread B requests UserContext → gets Instance B.

They do **not** share the same object.

11) What is a Constructor?

A constructor is a special method with the same name as the class and no return type. It is automatically called when an object is created to initialize the object's state.

12) Can we have multiple constructors in a class?

Yes, a class can have multiple constructors. This is known as constructor overloading. The constructors must differ in the number, type, or order of parameters.

13) Why can't constructors be overridden?

Constructors cannot be overridden because they are not inherited by subclasses. Overriding only applies to inherited methods. Each class has its own constructor, and when a child object is created, the parent constructor is invoked first (using super()), followed by the child constructor.

14) Have you worked on Low-Level Design (LLD)

Singleton

One instance shared per Spring container , Spring creates only one object of a singleton bean when the application starts. Whenever any controller, service, or repository needs that bean, Spring injects the same instance instead of creating a new one

15) Explain the Factory Design Pattern

The **Factory Design Pattern** is a **Creational Design Pattern** that provides an interface for creating objects **without exposing the object creation logic to the client**. Instead of using new directly, the client asks the factory to create the required object.

I have used factory when different implementation is needed based on the business logic . Because client does not need to know which concrete class get instantiated.

Q. Why use Factory instead of new?

Because the client should not know **which concrete class** is being created. The factory centralizes creation logic and makes it easier to change implementations later.

Q. Where have you used it?

A strong practical answer is:

- Spring Boot uses the pattern internally when creating beans through the Spring Framework IoC container.
- Database connection providers can return different implementations based on configuration.
- Payment gateways can return different processors (Credit Card, UPI, PayPal, etc.) based on the payment type.

"Where have you seen Factory and Singleton in Spring Boot?"

A strong answer is:

- **Factory Pattern:** "Spring's BeanFactory and ApplicationContext create and manage bean instances, so they are examples of the Factory pattern."
- **Singleton Pattern:** "By default, Sprining beans are singleton-scoped, meaning one bean instance is created per IoC container and shared across the application. This is similar to the Singleton pattern, although it's managed by Spring rather than by a private constructor and getInstance() method."

What is the Builder Pattern?

The **Builder Pattern** is a **Creational Design Pattern** used to construct complex objects step by step.

Instead of using a constructor with many parameters, we use a builder.

Purpose: Create complex objects step by step.

Example:

```
Employee emp = new Employee.Builder()
    .name("Nasima")
    .email("abc@gmail.com")
    .build();
```

Real-world

- Lombok @Builder
- HttpRequest.Builder

"Factory and Strategy look the same. What's the difference?"

A good answer is:

"Both use interfaces and polymorphism, but their purpose is different. The Factory pattern is responsible for creating and returning the appropriate object based on some input, hiding the object creation logic. The Strategy pattern assumes the object already exists and focuses on selecting and executing the appropriate algorithm or behavior at runtime. In many real-world applications, a Factory is used to create a Strategy, and then the Strategy is used to perform the business logic."

What is the Strategy Pattern?

The **Strategy Pattern** is a **behavioral design pattern** that allows you to **choose an algorithm or behavior at runtime**.

Instead of writing many if-else or switch statements, you put each algorithm in a separate class.

Advantages

- Removes long if-else chains.
- Easy to add new algorithms.
- Follows the **Open/Closed Principle**.
- Promotes loose coupling.
- Easy to test.

Observer pattern

The **Observer Design Pattern** is a behavioral design pattern where **one object (Subject)** notifies **multiple dependent objects (Observers)** automatically whenever its state changes.

Think of it as a **publish-subscribe mechanism**.

In Spring Boot, it's commonly implemented using ApplicationEventPublisher to publish events and @EventListener to react to them. This helps keep components loosely coupled because the publisher doesn't need to know which listeners will handle the event."

Strangler Pattern

The **Strangler Pattern (Strangler Fig Pattern)** is a migration pattern used to **gradually replace a monolithic application with microservices** without rewriting everything at once.

Example

Suppose an e-commerce application has these modules:

- User
- Order
- Payment
- Inventory

Instead of rewriting everything:

1. Move **User** module to a microservice.
2. Route /users/** requests to the new service.
3. Keep all other requests going to the monolith.
4. Repeat for Order, Payment, and Inventory.

Method in completable future

1.runAsync

runAsync is used to create a CompletableFuture that does not return a value.

2.supplyAsync

supplyAsync is used to create a CompletableFuture that returns a value

thenApply:

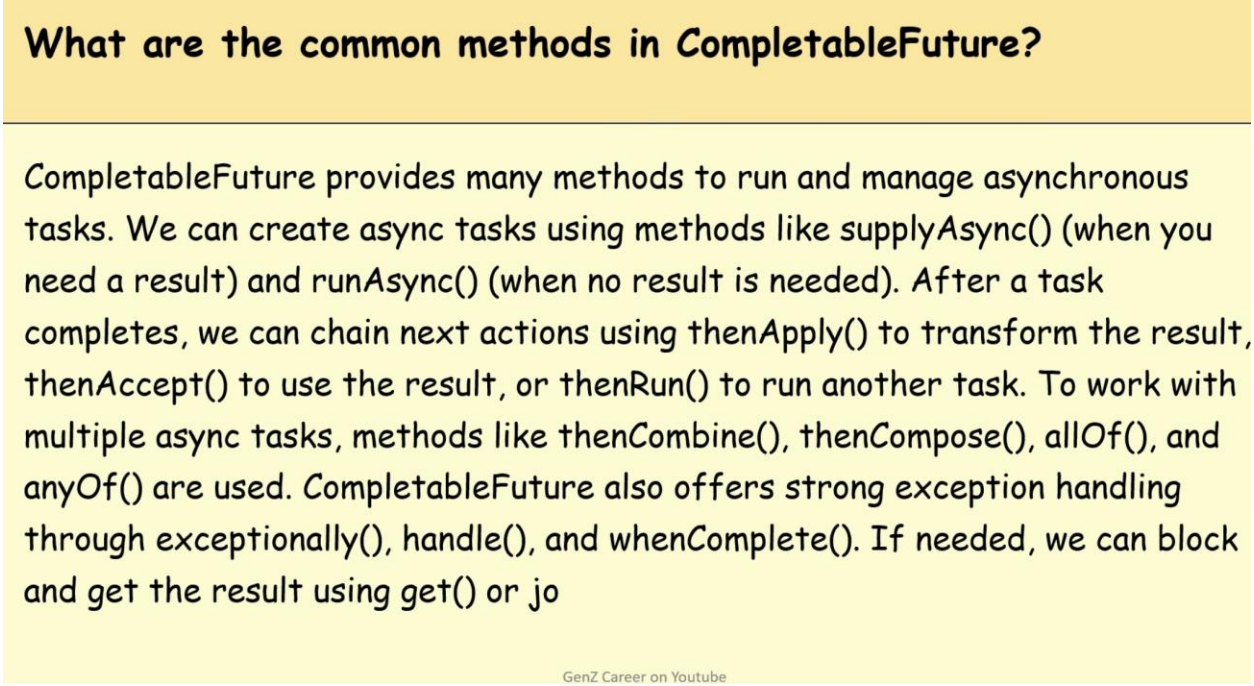
transform the result synchronously. This method is used when we want to apply some transformation on the result of the task.

thenApplyAsync:

transform the result asynchronously. This method is used when we want to apply some transformation on the result of the task asynchronously.

thenCompose: flatten nested futures (think async flatMap). This method is used when we want to chain multiple CompletableFuture and one task depends on the result of the previous task.

thenAccept / thenRun: consume results without returning a value.



16) Explain the Observer Design Pattern

The Observer Design Pattern is a behavioral design pattern that defines a one-to-many relationship between a subject and its observers. When the subject's state changes, it automatically notifies all registered observers. This pattern promotes loose coupling because the subject doesn't need to know the implementation details of its observers. Common examples include notification systems, event handling, and Spring's application event mechanism.

20) Why do we use @Qualifier with @Autowired?

@Qualifier is used with @Autowired when multiple beans of the same type are available in the Spring container. It tells Spring exactly which bean should be injected, avoiding a NoUniqueBeanDefinitionException.

For example, if an interface has CreditCardPayment and UpiP SQL ayment implementations, @Qualifier("upiPayment") ensures that the UpiPayment bean is injected.

@Primary can be used to define a default bean,

@Qualifier is used to explicitly choose a specific bean.

2. What is the `var` keyword? Can we use it as a generic type?

If we use var in varible type The compiler determines the type at **compile time**..

3. What is an effective final variable in Java?

An effectively final variable is a local variable that is not explicitly declared with the final keyword but whose value is never changed after initialization.

4. What is the Record class, and how do we store data in the Record class if a list of data is coming through a query? How to fetch the first record?.

Records are commonly used as **DTOs** in Spring Boot because they reduce boilerplate code while remaining immutable.

7. What is exception handling, and what is a Controller Advisor?

Exception handling is the process of handling runtime errors gracefully so that the application doesn't crash and instead returns a meaningful response.

@ControllerAdvice is a **global exception handler**.

Instead of writing @ExceptionHandler in every controller, we write it once in a class annotated with @ControllerAdvice, and it handles exceptions from all controllers.

9. Can we keep HashMap keys mutable? Why and why not?

Yes, you can use mutable objects as HashMap keys, but you should not.

The reason is that if the key changes after being inserted, the HashMap may no longer be able to find it.

Why?

HashMap uses two methods internally:

- hashCode() → Determines which bucket the key belongs to.
- equals() → Compares keys within the bucket.

11. How many types of methods can we write in an interface?

Default,static,public.

12. What happens if we write two identical default methods in two different interfaces and try to implement them in a class? How to fix the issue?

if two interfaces have **default methods with the same signature**, and a class implements both interfaces, the compiler doesn't know which default implementation to use.

How to fix it?

You **must override** the conflicting method in the implementing class.

You get a **compile-time error**.

Calling a specific interface's default method

If you want to reuse one of the interface implementations, use:

```
class Test implements A, B {

   @Override
   public void show() {
      A.super.show();   // Calls A's default method
   }
}
```

14. What are Liskov's substitutional principles? If we can replace the parent class object with a subclass, then why do we need inheritance?

Inheritance is **not created because we want substitution**.

Inheritance is used to model an **"is-a" relationship** and to **reuse common behavior**.

LSP simply tells us:

If we decide to use inheritance, the subclass must truly behave like the parent.

16. Diff between synchronized keyword and ReentrantLock. What is the trade-off?

synchronized is a JVM-level keyword that automatically acquires and releases a lock, making it simple and less error-prone. ReentrantLock is a lock implementation from java.util.concurrent that provides advanced features like tryLock(), timed lock acquisition, interruptible locking, fairness, and multiple Condition objects. The trade-off is simplicity versus flexibility: for most business applications, synchronized is sufficient and easier to maintain. I would choose ReentrantLock only when I need its advanced concurrency features."

When to use which?

- Use **synchronized** when:
  - You need simple thread safety.
  - Locking is straightforward.
  - You don't need timeout, fairness, or interruptible locking.
- Use **ReentrantLock** when:
  - You need tryLock(), timed lock acquisition, or interruptible locking.
  - You require fairness.
  - You need multiple Condition variables.
  - You are implementing more complex concurrent algorithms.

20. What is the difference between a stream and a parallel stream? Have you used a parallel stream in a real project?

A sequential stream processes elements one by one using a single thread, whereas a parallel stream splits the data into multiple chunks and processes them concurrently using the ForkJoinPool. Parallel streams can improve performance for large, CPU-intensive, independent operations, but they introduce thread-management overhead and require thread-safe processing.

28. What is a spring profile, and how did you use it?

A Spring Profile is a way to separate configuration for different environments like net, testing, and production. We create profile-specific property files such as application-dev.properties and application-prod.properties, and activate the required profile using spring.profiles.active. Spring then loads the appropriate configuration automatically.

n (Heap, Stack, Metaspace)

5. How do you make a class thread-safe? Give a real time scenario.

```
class BankAcount{
   int balance =100;
   public void synchonized withdraw(int amount){
      balance=banan-amount;
   }
}
```

Deloitte

What does `System.out.println()` do internally?

- Abstract class vs Interface – can both have constructors?

- ✅ **Abstract class can have constructors.**
- ❌ **Interface cannot have constructors.**

Why can an Abstract Class have a constructor?

Although you **cannot create an object of an abstract class**, its constructor is called when a subclass object is created.

- Can overridden methods change access specifiers, return types, or exceptions?

But there are some rules if method was protected you can change it to public but reverse would not work. Child throws same exception

```
class Child extends Parent {

   @Override
   void read() throws Exception {

   }
}
```

Child throws subclass exception

```
class Child extends Parent {

   @Override
   void read() throws IOException {

   }
}
```

IOException is a subclass of Exception.

Unchecked exceptions (subclasses of RuntimeException) have no such restriction.

- What happens if you declare a method as static while overriding?

You cannot override a static method.

If a subclass declares a static method with the same signature as the parent, it is **method hiding**, not method overriding.

- Trick code with try-catch-finally:

```
try { return 3; }
catch(Throwable e) { return 5; }
finally { return 7; }
```

Will it compile? What will it return? Is catching Throwable a good practice?

Yes, the code compiles. It returns 7 because the finally block always executes before the method returns, and a return statement inside finally overrides any previous return from the try or catch block. Catching Throwable is generally considered bad practice because it also catches JVM Errors such as OutOfMemoryError and StackOverflowError, which applications typically should not attempt to recover from.

Best Practice

- ✅ Use finally for cleanup (closing files, releasing resources).
- ❌ Avoid return statements inside finally because they hide earlier return values and can even suppress exceptions, making debugging difficult.

Constructor chaining example: why does JVM insert super()?

Constructor chaining is the process where one constructor calls another.

In inheritance, if a child constructor doesn't explicitly call super() or super(arguments), the Java compiler automatically inserts super() as the first statement. This ensures the parent class is initialized before the child class because the child object includes the parent object's state. If the parent class doesn't have a no-argument constructor, we must explicitly call one of its available constructors using super(arguments)

Polymorphism scenario with casting:

```
Copy code
A a1 = new B();
B a2 = new A();
```

A request fails somewhere across 5 microservices. How will you debug?

First, we should start by tracking the request end-to-end. Then Use a request ID so we can follow the same request across all 5 microservices in logs. Check logs step-by-step to see where it first failed.

Then, check each service one by one:

- Look at logs, errors, and response codes
- Verify if any service is slow, down, or returning wrong data
- Check dependencies like database, API calls, or queues

Finally, use tools like distributed tracing (e.g., Zipkin, Jaeger) and monitoring dashboards to quickly spot the failing service. Once found, fix the issue and add alerts to catch it earlier next time.

GenZ Career on Youtube

Let say Payment API is called twice accidentally. How will you ensure duplicate payment doesn't happen?

We should Use Idempotency here, basically.

- Send a unique transaction ID with each payment request.
- If same request comes again, system checks ID and ignores duplicate.
- Store request status to prevent double processing.

1. What is blocking queue and when can we use it.

A BlockingQueue is a thread-safe queue used for communication between producer and consumer threads. If the queue is empty, a consumer calling take() blocks until an element becomes available. If the queue is full, a producer calling put() blocks until space is available. It eliminates the need for manual synchronization using wait() and notify(). Common use cases include producer-consumer systems, thread pools, asynchronous logging, and message-processing applications.

Rules to Make a Class Immutable

1. ✅ Declare the class as final (cannot be extended).

2. ✅ Make all fields private and final.
3. ✅ Initialize fields through the constructor.
4. ✅ Do not provide setter methods.
5. ✅ If a field is mutable (like List or Date), return a defensive copy or an unmodifiable view.

2. How to make a custom exception class immutable.

Why is this immutable?

- ✅ Class is final (cannot be subclassed).
- ✅ Field age is private final.
- ✅ Value is set only in the constructor.
- ✅ No setter methods.

8. Cascading type used in transactions.

Cascade defines what should happen to **related entities** when an operation is performed on the parent entity.

10. How Https work internally.

11. What is CORS

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls whether a web page can make requests to a different origin (domain, protocol, or port)..

12. What is pass by value and pass by reference .

Pass-by-Value

Pass-by-value means:

A copy of the variable is passed to the method.

Changes to the copied variable do not affect the original variable. There is **no pass-by-reference in Java**.

12. How do you handle concurrent updates to the same database row?

@Version is an annotation that tells Hibernate to maintain a **version number** for a database row.

Whenever the row is updated successfully, Hibernate **automatically increments the version**.

Solution 3: Transactions

Use:

@Transactional

to ensure all database operations succeed or fail together.

14. How do you optimize a slow SQL query?

When I find a slow SQL query, I first identify the bottleneck using the execution plan (EXPLAIN). Then I check whether the appropriate indexes exist and whether the query is scanning the entire table. I optimize the query by selecting only the required columns, avoiding unnecessary joins and subqueries, and ensuring filtering happens efficiently. I also review database statistics, consider pagination for large result sets, and monitor the query execution time before and after the changes.

Steps to Optimize a Slow SQL Query

1. Check the Execution Plan (EXPLAIN) ⭐ (Most Important)

```
EXPLAIN
SELECT * FROM employee
WHERE department = 'IT';
```

It shows:

- Index usage
- Full table scan
- Join strategy
- Estimated rows

This is the first thing to check.

2. Add Proper Indexes

Without an index:

```
SELECT * FROM employee
WHERE email = 'abc@test.com';
```

Database checks every row (Full Table Scan).

With an index:

CREATE INDEX idx_email
ON employee(email);

The database can find the row much faster.

If I find a slow query, first I would analyze its execution plan using EXPLAIN ANALYZE to identify the bottleneck.

Then I would check whether proper indexes are available, avoid SELECT *, retrieve only the required columns, and optimize unnecessary joins and subqueries.

If the query returns a large number of records, I would implement pagination.

Since we use Spring Boot and Hibernate, I would also check for N+1 query problems, unnecessary database calls and connection pool issues.

Finally, I would compare the query execution time before and after optimization to verify the improvement.

19. How would you Dockerize a Spring Boot application?

To Dockerize a Spring Boot application, I first generate the executable JAR using Maven. Then I create a Dockerfile using a Java 17 runtime image, copy the JAR into the container, expose port 8080, and start the application with java -jar. I build the image using docker build and run it with docker run -p 8080:8080

20. If two microservices fail during communication, how do you handle fault tolerance?

To handle failures during inter-service communication, I use a combination of **timeouts, retries, circuit breakers, fallbacks, and monitoring**. f a service keeps failing, a circuit breaker opens and prevents repeated calls, allowing the system to fail fast. I also implement fallback methods to return default responses, or meaningful error messages.

1️⃣ What are the key features introduced in Java 8 and Java 17?

Java 8 introduced functional programming features such as Lambda Expressions, the Stream API, Functional Interfaces, Method References, Optional, the new Date & Time API, default and static methods in interfaces, and CompletableFuture for asynchronous programming.

Java 17, which is an LTS release, introduced features like **Records** to reduce boilerplate code, **Sealed Classes** to restrict inheritance, **Pattern Matching for instanceof** to eliminate explicit casting, **Enhanced Switch Expressions** for cleaner syntax, **Text Blocks**

for multiline strings, and improved NullPointerException messages.

Explain Spring Bean scopes.

2. Prototype Scope ★ ★ ★

A new object is created every time the bean is requested.

3. Request Scope

Only for **web applications**.

One bean instance is created **per HTTP request**.

Session Scope

One bean instance per **HTTP session**

How can you improve Spring Boot application startup performance.

To improve Spring Boot startup performance, I reduce the number of beans created at startup, enable lazy initialization where appropriate, optimize component scanning, avoid unnecessary auto-configurations, and ensure database or external service initialization doesn't block startup.

Microservices
What is Heartbeat & Self-Preservation mode in Eureka?
How do you secure Spring Boot Actuator endpoints?
How do you handle distributed transactions in Microservices?

If the producer sends a message successfully but fails afterward, how does the consumer ensure reliable consumption?

How do you use JUnit and Mockito in your project?

In my project, I use **JUnit 5** for writing unit test cases and **Mockito** to mock dependencies. I mainly write unit tests for the service layer because it contains the business logic. I mock repositories and external services so the tests are isolated and don't depend on a real database or APIs

@Mock UserRepository repository;

@InjectMocks UserService service;

Have you worked with CI/CD pipelines?

5 What is the difference between Mock and Static Mock in JUnit?

A **Mock** is used to mock an object's instance methods

A **Static Mock** is used to mock static methods

8 Write code examples demonstrating:
Aggregation
Association
Composition

Association is a general relationship between two independent objects. For example, a Teacher teaches a Student, but both can exist independently.

Aggregation is a weak **Has-A** relationship where one object contains another, but the child object can exist independently. For example, a Department has Employees; if the Department is deleted, the Employees still exist.

Composition is a strong **Has-A** relationship where the child object's lifecycle depends on the parent. For example, a Car contains an Engine. When the Car is destroyed, the Engine is destroyed along with it.

10 What is the purpose of the @Repository annotation in Spring? What happens if it is not used?

@Repository marks a class as the persistence or DAO layer. It is a specialized form of @Component, so Spring detects it during component scanning and registers it as a bean. Its additional advantage is automatic translation of database-specific exceptions into Spring's DataAccessException hierarchy, making exception handling consistent across different databases. If we don't use @Repository or another bean registration mechanism, the class won't be managed by Spring and dependency injection will fail. If we use @Component instead, the bean is created, but we lose the automatic exception translation feature.

What are default methods in Java? Why were they introduced?
- What is the @FunctionalInterface annotation?-

If "String A = "deepak"" and later "A = "pranav"", what happens internally?

Explain with respect to the String Constant Pool (SCP).
- Different types of exceptions in Java. Explain them.
- Difference between ArrayList and Vector. (Is Vector still relevant?)

9 Implement an LRU (Least Recently Used) Cache.

What is Connection Pooling? Explain its lifecycle.

2️⃣ What problem does the Stream API solve, and why was it introduced?

4️⃣ Explain the difference between Imperative Programming and Functional Programming.

Swap two variables without using a third variable.

How are Spring Beans Singleton by default.

13. Explain ACID properties with a banking transaction example.

- Difference between method overloading vs overriding.

TCS - group employees by department
Cognizant - sort employees by empId
Mastercard - group employees into age ranges like 20-30 and 30-40, and keep it generic
Accenture - a Stream API coding question and an output question

Four companies, one topic: Java 8 streams. Three of them on an Employee list. That's what candidates got in their Java rounds.

The first version is easy. But be ready for the variations:

1. Count per department instead of the full list.
2. Only the names per department.
3. The highest-paid employee in each department.
4. Sort by salary, highest first, then by name.
5. Age ranges like 20-30 and 30-40, without hardcoding the ranges.
6. What does Stream.of(1, 2, 3).peek(System.out::println).count() print?

4. What causes OutOfMemoryError in production? How would you debug it?
- If multiple threads are using the same key-value pairs, which collection would you use: Hashtable, HashMap, or ConcurrentHashMap?
- Find the sum of all even numbers in a list using the Stream API.

🔹 Spring Boot & Frameworks

- Advantages of Spring Boot over Spring Framework.
- Explain the @SpringBootApplication annotation.
- What is Spring Actuator?
- Bean scopes in Spring. Explain Prototype and Request scope.
- What is the @Transactional annotation?
- Difference between @OneToOne and @OneToMany mappings.

🔹 SQL & Database

- What is Apache Kafka used for?
- What is a Kafka topic?

Real-world Scenarios

- How do you handle a production issue if the same issue is reported by multiple users?

1.How do you manage and secure API endpoints in a microservices architecture?

In a microservices architecture, API endpoints are managed using an API Gateway which handles routing, authentication, and rate limiting. Security is implemented using authentication mechanisms like JWT or OAuth2, followed by authorization using role-based access control.

Accenture : what is publisher and subscriber, diff between otp and token, autorization vs authentaication, arraylist vs linked,design pattern, solid principle,hasmap,hashet inter working, when to use thread and running, notify vs notify all,bean vs configuration

Which design patterns have you implemented in microservices?
What are the states of a Circuit Breaker?

Q1. How would you implement a thread-safe in-memory cache without using third-party libraries?
Q2. What problems can occur when using parallel streams in production systems?

***) Round 1: Core Java & DSA :-**

.) HashMap internals, collisions and resizing
.) Longest substring without repeating characters
.) Cycle detection in a linked list
.) Multithreading and shared objects
.) equals() vs hashCode()

***) Round 2: Spring Boot & Microservices**

.) Internal working of @Transactional
.) Self-invocation issues
.) Idempotent REST APIs
.) Handling downstream failures
.) Troubleshooting slow APIs

***) Round 3: System Design**

.) Highly available payment systems
.) Microservices data consistency
.) Memory leaks and thread pool exhaustion

.) Handling thousands of concurrent requests
.) Kafka vs synchronous REST

***) Round 4: HR & Managerial**
.) Challenging production issue
.) Why JP Morgan?
.) Handling technical disagreements
.) Long-term career goals

Q3. How does virtual thread scheduling work, and when would you avoid using virtual threads?
Q4. What are memory visibility issues that still exist even after using ConcurrentHashMap?
Q5. How would you investigate a Java application whose latency suddenly increased after a deployment?
Q6. How do you detect thread contention in a running production system?
Q7. ExecutorService vs manually creating threads?
Q8. How do Java Streams work internally?
Q9. What is lazy evaluation in Streams?
Q10. HashMap vs ConcurrentHashMap?
Q11. How does ConcurrentHashMap achieve thread safety?
Q12. How would you design an immutable class?
Q13. String vs StringBuilder vs StringBuffer?
Q14. Explain JVM memory areas.
Q15. Design a flash-sale system capable of handling millions of users simultaneously.
Q16. Design and implement a Time-Based Key Value Store.
Q17. What is Serialize and deserialize a binary tree.
Q18. How would you identify the root cause of a slow query in production?
Q9. What strategies can be used to avoid database hot spots?
Q20. How would you design pagination for a table containing billions of records?
Q21. How would you migrate a large database schema with zero downtime?
Q22. How do filters, interceptors, and API gateways complement each other?
Q23. How would you handle 50,000 requests per second on a Spring Boot service?

CORE JAVA:
1. HashMap internal working. What happens when a bucket turns into a tree.
2. HashMap vs ConcurrentHashMap vs Hashtable.
3. equals and hashCode contract. Break it and watch your Set duplicate.
4. Why is String immutable.
5. Comparable vs Comparator.
6. Checked vs unchecked exceptions.
7. Can you override a static method.
8. try with resources.
9. Which Java 8, 11 and 17 features you have actually shipped.

STREAMS:
10. Find duplicate strings in a list using Streams.

11. map vs flatMap.
12. Sort a map by value.
13. Optional. Where it actually saves you.

MULTITHREADING:
14. Runnable vs Callable.
15. ExecutorService and how you size a thread pool.
16. synchronized vs ReentrantLock.
17. volatile vs AtomicInteger.
18. Deadlock. How you detect it in a running JVM.

SPRING BOOT:
19. Spring vs Spring Boot.
20. How autoconfiguration works.
21. Component vs Service vs Repository.
22. How dependency injection resolves at startup.
23. Transactional. What rolls back on a checked exception.
24. ControllerAdvice and global exception handling.
25. Spring Security and the JWT flow.

MICROSERVICE:
26. How two services find each other.
27. What an API gateway actually does.
28. Circuit breaker. Why retries alone make the outage worse.
29. Tracing one request across five services.

SQL AND HIBERNATE:
30. When an index hurts instead of helps.
31. Find duplicate rows.
32. Second highest salary without LIMIT.
33. N plus 1 problem. How you kill it.
34. Lazy vs eager loading.

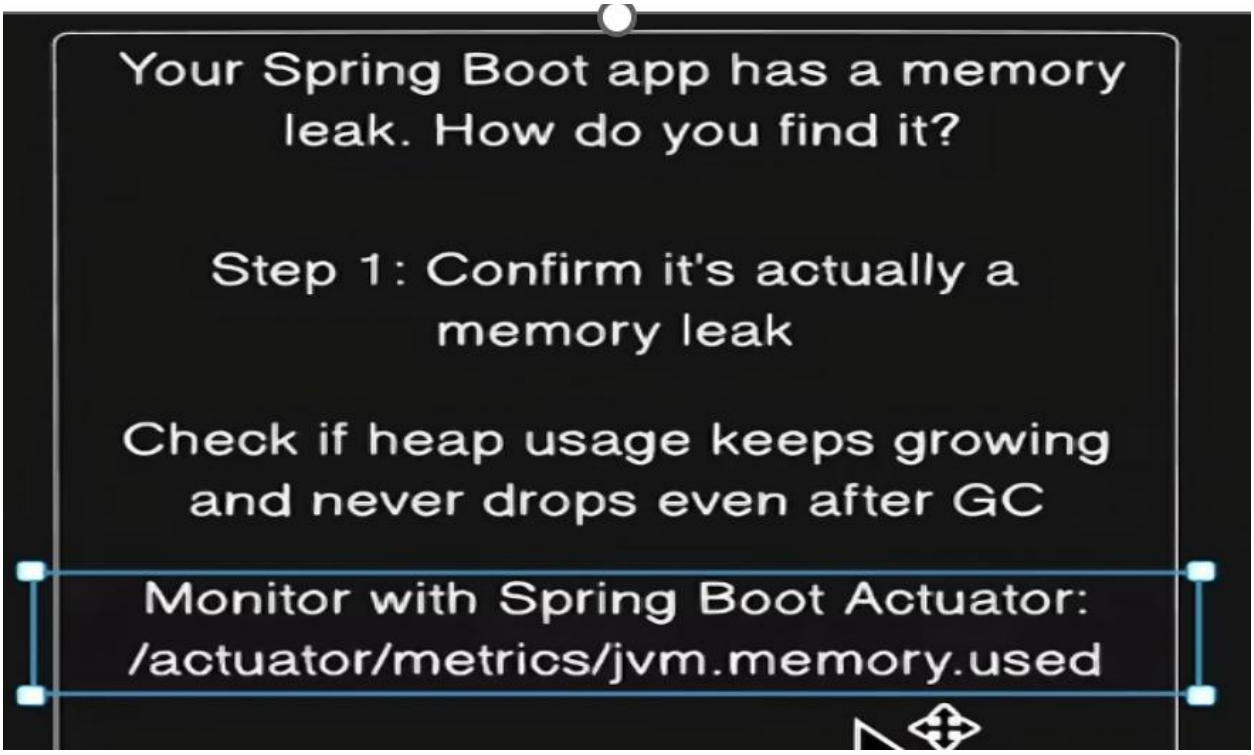
CODING ROUND:
35. Move all zeroes to the end.
36. Check if two strings are anagrams.
37. Longest common prefix.
38. Missing number in a consecutive array.

MANAGERIAL:
39. Draw your project architecture end to end.
40. A production bug you owned.
41. Notice period and relocation.

2. How do you identify and fix memory issues in production?

Ans.

We can take a heap dump using tools like jmap or jcmd by providing the process ID



Ans : I identify memory issues using monitoring and heap analysis, find root causes like memory leaks or large objects, and fix them by optimizing code and tuning JVM.

3.How would you handle 1 lakh (100K) concurrent requests?

In microservices architecture, we can scale individual services independently based on load. I would handle high concurrent requests using load balancing, horizontal scaling, caching, asynchronous processing, and database optimization.

4. How to switch from a monolithic architecture to a microservice architecture?

To migrate from a monolithic architecture to microservices, I would first analyze the existing system and identify service boundaries based on business capabilities. Then, I would gradually extract modules using the Strangler pattern instead of a complete rewrite. Each microservice would be independently developed with its own database. I would introduce an API Gateway for routing and security, implement communication using REST or messaging systems like Kafka, and use service discovery for dynamic routing. Finally, I would containerize services using Docker and deploy them using Kubernetes with proper monitoring and logging.

1. How do you develop a Spring Boot application and design microservices?

✔ Answer:

I start by creating a Spring Boot project using starters like web and data JPA. Then I follow a layered architecture with Controller, Service, and Repository. For microservices, I break the application into small services based on business capabilities, ensure each service has its own database, and expose REST APIs. I also use an API Gateway for routing and handle communication using REST or messaging.

5. How would you design a thread-safe class? Illustrate with a real-world scenario.

To design a thread-safe class, I ensure proper synchronization of shared resources using techniques like synchronized methods, atomic variables, or concurrent collections. I also try to minimize shared mutable state or use immutability where possible. For example, in a bank account system, I would synchronize deposit and withdraw methods to prevent race conditions when multiple threads access the balance.

6. How do you secure your application?

I secure my application using a layered approach. I implement authentication and authorization using JWT and role-based access control. I use an API Gateway to validate requests and centralize security. Communication is secured using HTTPS, and I ensure proper input validation to prevent attacks. I also implement rate limiting, proper exception handling, and logging to monitor and detect any suspicious activity.

7. Walk me through your architecture”

In my current project, we follow a microservices-based architecture in the telecom domain. The system is designed to monitor and manage tower-related issues remotely.

At a high level, the client applications send requests which are routed through an API Gateway, for example using Spring Cloud Gateway. The gateway handles routing, authentication, and basic request validation.

Behind the gateway, we have multiple microservices such as user service, monitoring service, and alert service. Each service is independently developed and deployed, and communicates with other services using REST APIs or asynchronous messaging where needed.

We also use service discovery, like Netflix Eureka, so that services can dynamically locate each other instead of hardcoding URLs.

Each microservice has its own database to ensure loose coupling and better scalability.

For deployment, we containerize our services using Docker and deploy them on Kubernetes, which helps with scaling and managing the application.

We also implement logging and monitoring to track system health, and use circuit breakers to handle failures gracefully.

Overall, this architecture helps us build a scalable, fault-tolerant, and maintainable system.'

8. If you want to send/receive data in XML instead of JSON, how will you do it?

We can enable XML support in Spring Boot by adding Jackson XML dependency and configuring the controller with consumes and produces as application/xml. Spring uses message converters to automatically serialize and deserialize XML requests and responses.

1. Your Spring Boot API suddenly becomes slow. How would you find the bottleneck?

2. 5,000 requests hit your API simultaneously. What could become the bottleneck?

3. Your database connection pool is exhausted. What would you check first?

4. A downstream service takes 10 seconds to respond. How would you protect your API?

5. The same request reaches your API twice. How would you make it idempotent?

6. Two users update the same record simultaneously. How would you handle it?

7. Your API works locally but fails in production. What would you investigate?

8. A "@Transactional" operation partially fails. Why might the rollback not happen as expected?

9. Your Java application suddenly consumes too much memory. How would you debug it?

10. A Microservice goes down while other services are healthy. How should your system handle it?

12. Your API becomes slow only during peak traffic. Which metrics would you check?

13. A retry mechanism is making a downstream outage worse. Why?

14. A scheduled job runs twice after scaling your application. How would you prevent it?

15. One request passes through multiple Microservices. How would you trace it end-to-end?
Explain the classloader hierarchy.
Describe the internal working of the JVM.
Write Java code to serialize and deserialize an object.
Implement a thread-safe cache in Java.
Can we inherit overridden and overloaded methods?
How do parallel streams work? Explain intermediate and terminal operations.

our Java app slows down over time without errors. What could be happening internally?

2) CPU is low but response time is high. What might be blocking threads?
3) OutOfMemoryError occurs even with sufficient heap. Why?
4) Increasing heap size made performance worse. Explain.
5) Threads are free but requests are queued. Why?
6) GC pauses increased after a small change. What might have changed?
7) JVM doesn't exit after main() completes. What's holding it?
8) Parallel streams reduced performance instead of improving it. Why?
9) Memory usage keeps increasing slowly. What do you suspect?
10) Logging change caused slowdown. How?
11) ThreadLocal fixed one issue but caused another. Why?
12) ExecutorService tasks fail silently. What went wrong?
13) Retry logic overloaded the system. What mistake was made?
14) Deadlock happens only in production. Why?
15) Scaling instances made performance worse. Why?

Spring Boot (Production Scenarios)

16) App works locally but fails in production. What do you check first?
17) APIs are fast locally but slow in prod. Why?
18) **application.properties** change didn't reflect. Why?
19) CPU is low but requests timeout. What's the bottleneck?
20) Multiple beans cause startup failure. How to fix?
21) @Transactional present but rollback doesn't happen. Why?
22) DB connection pool exhausts under load. Why?
23) Scheduled jobs affect API latency. How to isolate?
24) App behaves differently in Docker vs local. Why?
25) New deployment but users see old behavior. What went wrong?

26) Logs missing in production but present locally. Where to check?
27) Async processing made performance worse. How?
28) Circuit breaker stays open even when service is healthy. Why?
29) Adding more resources didn't improve performance. Why?
30) What Spring Boot decision you made could cause production issues?
What design patterns are you familiar with?
Explain OOP principles.

How do you implement caching using Redis?
Is a List mutable or immutable? Why?
Is the Date class mutable? How can you make it immutable?
How does HashSet work internally?
What is the time complexity of HashSet operations?
What are the different ways to create an object in Java?
How does cloning work in Java?
How can you make a HashMap thread-safe?
💡 These questions will be really helpful for anyone preparing for Java & Microservices interviews.
Explain the SOLID Design pattern.
What is JWT security, and how have you used it in your project?
Stream vs flat Map.
Explain basic Spring Security.
Reverse a string without using predefined methods.
Spring Container internal working.
What is an Optional class?
What is the difference between @primary and @Qualifier?
Explain Completable Future methods.
Explain Bean Lifecycle.
Explain Maven Lifecycle.
What is the difference between Path Variable and Request Params?
What is fetch type (Lazy and Eager Loading)?

Java Backend Interview Questions (Must Prepare)

1️⃣ Can we perform an update operation using HTTP POST?
2️⃣ If two objects have the same hashCode(), what will be the result of equals()?
3️⃣ If equals() returns true, will their hashCode() always be the same?
4️⃣ How will you validate whether the same user is hitting the request or not using JWT?
5️⃣ How do you store passwords in an application? (Plain text or encrypted?)
6️⃣ What are the major changes in Java 11 & Java 17 in terms of performance?
7️⃣ What are the methods available in the Object class?
8️⃣ What are default methods in interfaces?
9️⃣ Explain ConcurrentHashMap and internal working of TreeSet.
🔟 What are Hibernate states? Explain lazy loading, get() vs load() methods.

1️⃣1️⃣ Find the missing element in an array.
1️⃣2️⃣ If two interfaces have the same method and a class implements both, how does it work?
1️⃣3️⃣ What do you mean by dynamically loading a class? Where is it used?
1️⃣4️⃣ How do you secure REST endpoints?
1️⃣5️⃣ Difference between Comparator and Comparable.
1️⃣6️⃣ Difference between Runnable and Callable.
1️⃣7️⃣ What are default methods?
1️⃣8️⃣ What is the volatile keyword?
1️⃣9️⃣ Before Java 8, what was used instead of default methods?
2️⃣0️⃣ Difference between abstract class and interface.
2️⃣1️⃣ Difference between terminal and intermediate operations in Streams. What is Functional Interface and Lambda Function?
Difference between Collections and Collection
Difference between HashMap and ConcurrentHashMap
Different ways to iterate a Map
How many null values are allowed in HashMap and ConcurrentHashMap?
Where do you use HashMap in your real project?
Difference between static and instance variable
Which static method executes (parent or child) when called from parent class? How does method binding work internally?
What is a class loader and what are different types of class loaders?
How microservices communicate (Synchronous vs Asynchronous communication)
What is HTTPS communication and difference between RestTemplate and WebClient
How indexing works internally
Explain and implement the factory design pattern (with some modification)
What do you know about ISO8583?
Diff b/w IOC and Dependency Injection.
What is Dependency Injection and its types?
Find the second-highest salary from the employee table.
Find a file in a subdirectory
How have you used Spring Security in your project?
What is a spring profile, and how did you use it?
What is exception handling, and what is a Controller Advisor?
How does the @Transactional annotation work?
Below are 20 questions candidates are being asked in Round 1:
1. Explain the internal working of HashMap, including hashing, buckets, collision handling, and resizing.
2. Differentiate between HashMap and ConcurrentHashMap in terms of concurrency control.
3. Describe the JVM memory model (Heap, Stack, Metaspace) and object allocation flow.
4. What are the key reasons for OutOfMemoryError in production, and how would you analyze it?

- Find nth highest salary.

Emp.stream().collect(Comparator.comparaing(Emp::getSalaru).reversed)).findFirst()

- Difference between Clustered and Non-Clustered Index.
- Explain database normalization.
- How would you optimize a slow query?

Project Discussion

- Explain one production issue you resolved.
- How do you avoid duplicate message processing?
- How do you scale a microservice handling high traffic?

6. Walk through the complete lifecycle of a Spring Bean.
7. How does Spring implement Dependency Injection internally?
8. What occurs behind the scenes when @Transactional is applied?

10. How would you configure centralized/global exception handling in Spring Boot?
11. Explain Lazy vs Eager fetching in JPA and associated performance trade-offs.
12. How do you handle concurrent modifications to a single database record?

To handle concurrent updates to the same database record, I typically use optimistic locking with JPA's @Version annotation because it works well when update conflicts are infrequent. If two users update the same record, the second update fails with an OptimisticLockException, preventing lost updates. I also use @Transactional to ensure atomicity and rely on appropriate database isolation levels based on the application's consistency and performance requirements.

1. Explain HashMap internal working.
2. How does hashing work internally?
3. How are collisions handled?
4. What happens during HashMap resizing (Rehashing)?
5. What is the default initial capacity and load factor?
6. What are Custom Exceptions in Java?
7. How do you handle exceptions in Spring Boot applications?
8. Predict the output of a Java code snippet involving Collections and Exception Handling.
9. Explain the output and identify the issue in the code.
10. What happens if a collection is modified while iterating?
11. How does ConcurrentModificationException occur?
12. Write a coding example demonstrating both Fail-Fast and Fail-Safe behavior.
13. Explain all five SOLID principles.
14. Which SOLID principles have you implemented in your project?
15. Explain ACID Properties with a banking transaction example.
16. How does rollback work?
17. What are the different states of Circuit Breaker?
18. Which library have you used for Circuit Breaker implementation?

Docker & CI/CD
19. Explain your CI/CD pipeline.
20. What are the stages in your deployment pipeline?
21. Which deployment strategy do you follow in your project?
22. Explain Blue-Green Deployment.
23. Why are Offsets important?
24. How do you avoid duplicate message processing?
25. What happens if a consumer crashes before committing the offset?

13. Explain ACID properties with a real-world banking transaction example.

15. How would you architect pagination and sorting in a RESTful API?
16. Explain the end-to-end workflow of JWT-based authentication.
17. If an API becomes slow under high traffic, how would you diagnose the root cause?
18. REST vs Event-Driven Messaging (Kafka) when and why would you choose one over the other?
19. Outline the steps to Dockerize a Spring Boot application for deployment.
20. How would you implement fault tolerance if two microservices fail during inter-service communication?

1. How does HashMap work internally? Explain buckets, hashing mechanism, and resizing process.
2. What is the difference between HashMap and ConcurrentHashMap?
3. Explain the JVM memory architecture (Heap, Stack, Metaspace).
4. What are common causes of OutOfMemoryError in production? How would you troubleshoot it?
5. How do you make a class thread-safe? Provide a practical example.
6. Describe the complete lifecycle of a Spring Bean.
7. How does Dependency Injection function internally in Spring?
8. What happens internally when you use @Transactional?
9. What is the difference between @Component, @Service, and @Repository?
10. How would you implement global exception handling in Spring Boot?
11. What is lazy vs eager loading in JPA? In which scenarios can it impact performance?
12. How do you manage concurrent updates to the same database record?
13. Explain ACID properties using a banking transaction example.
14. How would you optimize a slow-performing SQL query?
15. How would you design pagination and sorting in a REST API?
16. Explain the step-by-step flow of JWT authentication.
17. If your API performance degrades under load, how would you identify the bottleneck?
18. REST vs Messaging (Kafka) when would you choose each approach?
19. How would you Dockerize a Spring Boot application?
20. If two microservices fail during communication, how would you implement fault tolerance?

1) Java app slows down over time without errors. What's happening internally?
2) OutOfMemoryError occurs even when heap looks sufficient. How?
3) CPU is low but latency is high. What could be blocking?
4) Threads are available, yet requests are queued. Why?
5) Increasing heap size made performance worse. Explain.

6) GC pauses increased after a small code change. What changed?
7) JVM doesn't exit after main() finishes. What's holding it?
8) Parallel streams reduced throughput instead of improving it. Why?
9) Memory keeps growing without static references. What do you suspect?
10) Logging level change caused production slowdown. Why?
11) ThreadLocal fixed one issue but created another. How?
12) ExecutorService tasks fail silently. What went wrong?
13) App behaves differently on Java 8 vs Java 17. Why?
14) Retry logic overloaded the system. What was the mistake?
15) Scaling instances made things worse. Why?

Spring Boot (Production & Scenarios)

16) App works locally but fails after deployment. What do you check first?
17) APIs are fast locally but slow only in prod. How do you debug?
18) **application.properties** change didn't reflect. Why?
19) CPU is low but requests timeout. What's the bottleneck?
20) Multiple beans of same type cause startup failure. How to fix?
21) @Transactional exists but rollback doesn't happen. Why?
22) DB connection pool exhausts under load. Why?
23) Scheduled jobs impact API latency. How do you isolate?
24) App behaves differently in Docker vs local. Why?
25) New deployment but users see old behavior. What went wrong?
26) Logs missing in production but present locally. Where to check?
27) Async processing made performance worse. How?
28) Circuit breaker stays open even when service is healthy. Why?
29) Adding more resources didn't improve performance. Why?
30) What Spring Boot decision you made once caused a real production issue?

2. End-to-end flow of how an Entity class eventually becomes a table in the database (JPA → Hibernate → SQL)

= we create an entity class using @Entity @Table ,

JPA is just a specification, hibernate is the jpa implementation,Hibernate generates the entity metadata and generate the sql quesries , it also updates the table.

3. Why we use @Service and @Repository, what Spring does differently with them internally
4. What happens if both @Service and @Repository are used on the same class
5. Thread lifecycle - especially NEW vs RUNNABLE states

💡 Core Java – Important Interview Questions (with real understanding)

1. Why is Java platform independent? How does it actually work?

Java source code is compiled into bytecode by the compiler. This bytecode is not machine-specific and is executed by the JVM. Since every platform has its own JVM implementation, the same bytecode can run on any system.

👉 Real meaning: Write once, run anywhere because of JVM.

2. Difference between JDK, JRE, and JVM in real usage?

JVM: Executes bytecode
JRE: JVM + runtime libraries
JDK: JRE + development tools (compiler, debugger)

👉 Developer needs JDK, user needs only JRE.

3. What happens internally when we run a Java program?

Code is compiled → bytecode (.class)
ClassLoader loads class into JVM
Bytecode verified for security
Interpreter/JIT executes it

👉 This shows you understand Java execution flow.

4. What is OOP and why is it important in real projects?

OOP organizes code into objects, making systems modular, reusable, and easier to maintain.

Real benefit: scalability + clean architecture.

5. Difference between == and equals()?

== compares memory reference
equals() compares object content (logical equality)

7. What is memory management in Java?

Java uses Heap & Stack memory. Garbage Collector automatically removes unused objects from heap.

Shows understanding of runtime behavior.

8. What is Constructor and why is it needed?

Constructor initializes object state when object is created.

Real-world: dependency injection, object setup.

9. Difference between Array and ArrayList?

Array: fixed size
ArrayList: dynamic size, part of collections

👉 Very common in service-based companies.

10. What is Exception Handling and why is it important?

Exception handling prevents program crash and allows graceful error recovery.

Real-world: APIs, validations, production systems.
1. Why did Java adopt OOP instead of procedural programming?

OOP improves scalability, reusability, and maintainability by organizing code around real-world objects instead of functions.
👉 Interviewer checks: design mindset

2. Difference between Abstraction and Encapsulation (real-world view)?

Abstraction hides complexity and shows only required functionality
Encapsulation protects data by binding variables and methods together

One defines what to expose, the other how to protect.

3. Why is Java not 100% object-oriented?
Because Java uses primitive data types (int, boolean, etc.) for performance reasons.

Very commonly asked trap question.
4. Why does Java not support multiple
inheritance using classes?

To avoid ambiguity caused by the Diamond Problem, which can lead to method conflict.

5. How does Java still support multiple inheritance?

Through interfaces, because they avoid state conflict and support multiple implementation.

Framework-level concept (Spring heavily uses this).

6. Difference between compile-time and runtime polymorphism?

Compile-time: Method overloading
Runtime: Method overriding

Foundation for dynamic behavior in Java.

7. What is runtime polymorphism and where is it used in real projects?

Runtime polymorphism allows parent reference to call child implementation and is widely used in frameworks like Spring for loose coupling.

8. Why can't constructors be overridden?

Because constructors are not inherited and are used only to initialize objects.

9. Can we achieve abstraction without abstract classes or interfaces?

Yes, using access modifiers and method design, but Java provides abstract classes and interfaces for structured abstraction.

10. How does OOP help in large-scale applications?

1. What is the Java Collections Framework?

It is a unified architecture that provides interfaces, implementations, and algorithms to store and manipulate groups of objects efficiently.
👉 Checks basic clarity.

2. Difference between Collection and Collections?

Collection: Interface (List, Set, Queue)
Collections: Utility class with static methods (sort, reverse, etc.)
👉 Very common trap question.

3. Difference between Array and ArrayList?

Array: Fixed size, can store primitives
ArrayList: Dynamic size, stores objects only
👉 Performance + flexibility question.

4. Difference between ArrayList and LinkedList?

ArrayList: Faster random access
LinkedList: Faster insertion/deletion
👉 Asked in almost every interview.

5. How does HashMap work internally?

HashMap stores data as key–value pairs.
It uses hashing, where hashCode() determines the bucket and equals() resolves collisions.
👉 This is a must-know question.

6. What is load factor in HashMap?

Load factor decides when rehashing happens.
Default value = 0.75, balancing time and space complexity.

7. What happens when two keys have the same hashCode?

Collision occurs.
HashMap uses equals() to differentiate keys and stores them using linked list / tree (Java 8+).

8. Why should we override hashCode() and equals()?

To ensure correct key comparison and avoid duplicate keys in HashMap.
👉 Very important for real projects.

9. Difference between HashMap and Hashtable?

HashMap: Not synchronized, faster
Hashtable: Synchronized, legacy class

10. Difference between HashMap and ConcurrentHashMap?

ConcurrentHashMap is thread-safe and allows concurrent access without locking the entire map.
1. What is Multithreading and why is it needed?

Multithreading allows multiple threads to run concurrently, improving performance and better CPU utilization.

2. Difference between Process and Thread?

Process: Independent execution unit
Thread: Lightweight unit within a process
Threads share memory, processes do not.

3. Difference between Thread class and Runnable interface?

Thread: Extends class (no multiple inheritance)
Runnable: Better design, supports multiple inheritance
👉 Runnable is preferred in real projects.

4. What is synchronization and why is it required?

Synchronization prevents multiple threads from accessing shared resources simultaneously, avoiding data inconsistency.

5. What is a deadlock?

Deadlock occurs when two or more threads wait indefinitely for each other's resources.
👉 Very common interviewer favorite.

6. What is the volatile keyword?

volatile ensures visibility of variable changes across threads but does not provide synchronization.

7. Difference between wait() and sleep()?

wait(): Releases lock, used for inter-thread communication
sleep(): Does not release lock, pauses execution

8. What is Thread lifecycle?

New → Runnable → Running → Waiting/Blocked → Terminated

9. What is Executor framework?

It manages thread creation and execution efficiently instead of creating threads manually.

10. Why is multithreading important in backend applications?

Used for handling multiple requests, background tasks, and improving system performance.

➡️ Next: Part 5 – Exception Handling (Real-world scenarios)
1. What is an Exception in Java?

An exception is an unwanted or unexpected event that occurs during program execution and disrupts normal flow.

2. Difference between Checked and Unchecked Exceptions?

Checked: Checked at compile time (IOException, SQLException)
Unchecked: Checked at runtime (NullPointerException, ArithmeticException)
👉 Very frequently asked.

3. Difference between Exception and Error?

Exception: Can be handled by application
Error: Serious problem, usually not handled (OutOfMemoryError)

4. What is try–catch–finally block?

try: Code that may cause exception
catch: Handles exception
finally: Executes always (cleanup code)
👉 Real-world use: closing DB connections.

5. Can finally block execute without catch?

Yes, finally can be used with try block even without catch.
-
6. What is throw vs throws?

throw: Used to explicitly throw an exception
throws: Used to declare exceptions in method signature

7. What is custom exception and why do we use it?

Custom exceptions are user-defined exceptions used to represent business logic errors clearly.
👉 Common in Spring Boot applications.

8. Can we handle multiple exceptions in a single catch block?

Yes, using multi-catch (catch(IOException | SQLException e)).

9. What happens if an exception is not handled?

The program terminates abnormally and JVM prints stack trace.

10. Why is exception handling important in backend applications?

It prevents application crashes, improves reliability, and helps in graceful error handling.
1. What is String in Java?

String is an immutable object that represents a sequence of characters.

2. Why is String immutable in Java?

Because of security, thread-safety, caching in String Pool, and performance optimization.
👉 Very common why-based question.

3. Difference between String, StringBuilder, and StringBuffer?

String: Immutable, slow for modifications
StringBuilder: Mutable, not thread-safe, fast
StringBuffer: Mutable, thread-safe, slower

4. What is String Pool?

String Pool is a special memory area in heap that stores String literals to reduce memory usage.

5. Difference between == and equals() in String comparison?

== compares reference
equals() compares content
👉 Interview trap question.

6. What happens when we create String using new keyword?

A new object is created in heap memory even if the same value exists in String Pool.

7. Why are Strings widely used as keys in HashMap?

Because Strings are immutable and provide consistent hashCode(), making them reliable keys.

8. How does immutability help in multithreading?

Immutable objects are thread-safe because their state cannot be changed.

9. Why is StringBuilder faster than StringBuffer?

Because StringBuilder is not synchronized, avoiding thread overhead.

10. When should we use StringBuilder over String?

When frequent string modifications are required, such as loops or dynamic content.
1. What is JVM and what are its main responsibilities?

JVM executes bytecode, manages memory, performs garbage collection, and ensures platform independence.

2. What are the main memory areas in JVM?

- Heap (object storage)
- Stack (method calls & local variables)
- Method Area (class metadata)
- PC Register
- Native Method Stack

👉 Very frequently asked.

3. Difference between Heap and Stack?

Heap: Stores objects, shared among threads
Stack: Stores method calls, thread-specific

4. What is Garbage Collection?

Garbage Collection automatically removes unused objects from heap memory.

5. How does JVM identify objects for GC?

Objects without references (unreachable objects) are eligible for garbage collection.

6. What is the difference between Minor GC and Major GC?

Minor GC: Cleans Young Generation
Major GC: Cleans Old Generation

7. What is OutOfMemoryError?

Occurs when JVM cannot allocate memory in heap.

8. What is finalize() method? Is it recommended?

finalize() is called before GC removes object, but it is deprecated and not recommended.

9. What is memory leak in Java?

When objects are no longer needed but still referenced, preventing GC.

10. Why is understanding JVM important for backend developers?

Because performance tuning, scalability, and debugging production issues depend on memory understanding.
@RestController = @Controller + @ResponseBody
That's it. But what does that actually mean?

💡 Without @RestController
If you use only @Controller, Spring expects you to return a view (HTML page).

💡 With @RestController
Spring directly returns JSON (or any response body) instead of a view.
Now the response is: Hello World

Or if returning an object:
Spring automatically converts it to JSON using Jackson.

⚙️ What Happens Internally?
1. Spring scans the class
2. Maps HTTP requests via @GetMapping, @PostMapping
3. Converts return object → JSON using HttpMessageConverters
4. Sends response with proper Content-Type: application/json

🧠 When Should You Use It?
✅ REST APIs
✅ Microservices
✅ Backend for React / Angular
✅ Mobile app backend

Basically, if you're building an API → use @RestController.

🚨 Common Mistake
If you forget @RestController and use only @Controller,
you'll get weird errors like:
"Circular view path"
or Spring trying to find an HTML template
Happens to everyone at least once 😅

Q31: What is the difference between fail-fast and fail-safe iterators in Java?
Answer:
Fail-fast iterators immediately throw a ConcurrentModificationException if the collection is modified while iterating. Most collections like ArrayList use this mechanism.
Fail-safe iterators work on a cloned copy of the collection, so modifications do not throw exceptions. CopyOnWriteArrayList is an example.
Simple Real-life Example:
Fail-fast is like a strict teacher who stops the exam if someone cheats.
Fail-safe is like allowing changes quietly because you're working on a photocopy.

Q32: What happens internally when you use HashMap?
Answer:
When you insert a key-value pair, HashMap calculates the hash code of the key, converts it into an index, and stores the entry in a bucket.
If multiple keys map to the same bucket, collision handling happens using a linked list or tree structure (after Java 8).
Performance is O(1) on average but can degrade if hash collisions are high.
Simple Real-life Example:
Think of a HashMap like a library locker system. The locker number is decided using a formula (hash function). If two books get the same locker, they're placed inside in an organized way.

Q33: What is the difference between compile-time and runtime polymorphism?
Answer:
Compile-time polymorphism is achieved through method overloading and is decided at compile time.
Runtime polymorphism is achieved through method overriding and is decided at runtime using dynamic method dispatch.
Simple Real-life Example:
Compile-time polymorphism is like choosing your meal while ordering.
Runtime polymorphism is like the chef deciding how to cook it based on available ingredients.

Q34: What is the purpose of the volatile keyword in Java?

Answer:

The volatile keyword ensures visibility of changes to variables across threads. It prevents threads from caching variable values locally and guarantees that the latest value is always read from main memory. However, it does not provide atomicity.

Simple Real-life Example:

It's like a public notice board — once updated, everyone sees the latest version immediately.

volatile guarantees that threads see the latest value, but it does not make compound operations like count++ thread-safe. Atomic classes such as AtomicInteger guarantee both visibility and atomic updates, preventing race conditions.

Easy Memory Trick

- **volatile** → "Can all threads see the latest value?" ✅
- **AtomicInteger** → "Can all threads safely update the value?" ✅

1. Race Conditions

A race condition occurs when two or more threads try to read and modify a shared variable at the exact same time.

Race Condition vs. Deadlock

ute	Race Condition	Deadlock
Nature	Data corruption due to timing.	Total freeze due to resource dependency.
ary Cause	Unsynchronized access to shared variables.	Circular waiting for locks or resources.
m State	The program keeps running, but with wrong data.	The program hangs or stops responding entirely.
to Spot	Hard to replicate; often shows up as intermittent bugs.	Easy to spot; the application completely stops moving.

•••

Q35: What is the difference between Comparable and Comparator?
Answer:
Comparable is used to define the natural ordering of objects inside the class itself by implementing the compareTo() method.
Comparator is used to define custom sorting logic outside the class by implementing the compare() method.
Simple Real-life Example:
Comparable is like a person deciding their own ranking rules.
Comparator is like an external judge deciding how participants should be ranked.

Why use DTO when we already have Entity classes?
🚀 Entity → DTO → API Layer 🚀

While building my Spring Boot project APIs, I moved from returning Entity objects directly to using DTOs — and it significantly improved response control and API safety.

Why not expose Entity directly:
• Represents full DB table with all fields & relationships
• Can expose sensitive data and lazy-loaded relations
• DB changes directly affect API responses

Why use DTO:
• Sends only required fields to client
• Separate Request/Response models for better API design
• Better validation and payload control

Entity ↔ DTO mapping ways:
• Manual mapper methods
• Utility mapper classes
• Mapping libraries (ModelMapper / MapStruct)

In short:
Entity = persistence model
DTO = controlled API model

✓ Abstraction vs Encapsulation (real project example)
✓ Interface vs Abstract Class (Java 8+ changes)
✓ Multiple inheritance in Java
✓ Composition vs Inheritance

Inheritance represents an **IS-A** relationship where a child class extends a parent class and inherits its behavior. Composition represents a **HAS-A** relationship where one class contains an object of another class.

Inheritance provides code reuse but creates tight coupling, while composition promotes loose coupling and flexibility. In modern Java design, composition is generally preferred over inheritance unless there is a strong IS-A relationship.

- Reverse words in a String without using library methods.
- Find first non-repeating character.
- Find longest substring without repeating characters.
- Check if two strings are anagrams.
- LRU Cache design.
- Design Parking Lot (LLD discussion).

- Multi-threaded counter implementation.

Spring Boot

- Find nth highest salary.

Select salray from emp order by salaray desc limit 1 offset n-1;

- Difference between Clustered and Non-Clustered Index.
- Explain database normalization.

Normalization is the process of **organizing data to reduce redundancy and improve data integrity**.

- How would you optimize a slow query?

When optimizing a slow query, I first analyze the execution plan using EXPLAIN to identify bottlenecks. Then I check whether appropriate indexes exist, avoid SELECT *, retrieve only the required columns, optimize joins and WHERE conditions, ensure indexes are being used, and consider caching frequently accessed data

Project Discussion

- Explain one production issue you resolved.

- How do you avoid duplicate message processing?
- How do you scale a microservice handling high traffic?

✓ Static method overriding? Constructor overriding?
✓ Diamond Problem & Java's solution
✓ SOLID – SRP in real systems
Object Class (Favorite Trap Area)
✓ equals() vs ==
✓ hashCode() contract & HashMap impact
✓ Same hashCode ≠ equal objects?
✓ String immutability & String Pool
✓ String vs StringBuilder vs StringBuffer
Multithreading (Interview Killer ⚠️)
✓ Thread safety & memory visibility
✓ synchronized vs ReentrantLock
✓ volatile (real-world use case)
✓ wait() vs sleep()

"wait() is used for inter-thread communication. It releases the object's monitor lock and waits until another thread calls notify() or notifyAll(). sleep() simply pauses the current thread for a specified time but does not release any lock it holds."

2. notify() vs notifyAll()

Both methods belong to the **Object** class.

They wake up threads that are waiting using wait().

✓ Deadlock – detection & prevention
✓ Thread lifecycle
✓ Runnable vs Callable
✓ Future vs CompletableFuture

Future was introduced in Java 5 and represents the result of an asynchronous computation. However, it has limitations because get() blocks until the task completes, it doesn't support task chaining, combining multiple tasks, or advanced exception handling.

CompletableFuture, introduced in Java 8, extends these capabilities by allowing asynchronous callbacks, chaining operations with methods like thenApply, combining multiple futures using allOf and anyOf, and handling exceptions using methods such as exceptionally and handle. For modern Java and Spring Boot applications, CompletableFuture is generally preferred.

✓ Thread pool internals
✓ ExecutorService vs ForkJoinPool
Collections (Most Asked)
✓ HashMap internal working (Java 8)
✓ Why HashMap isn't thread-safe
✓ ConcurrentHashMap internals
✓ HashMap vs Hashtable vs ConcurrentHashMap
✓ Why only one null key in HashMap?
✓ TreeMap vs HashMap (sorting logic)
✓ Fail-fast vs Fail-safe iterators
✓ entrySet() vs keySet() performance

✓ Custom objects as Map keys
✓ Comparable vs Comparator (real use case)
Exception Handling
✓ Checked vs Unchecked exceptions
✓ try-catch-finally execution order
✓ Can finally be skipped?
✓ throw vs throws
✓ Custom exceptions – when & why
✓ Exception handling best practices in REST APIs
JVM & Memory (Senior Filter)
✓ JVM vs JRE vs JDK
✓ Stack vs Heap
✓ Metaspace internals
✓ Types of Garbage Collectors
✓ Minor GC vs Major GC
✓ Memory leak debugging
✓ OutOfMemoryError causes
✓ Why finalize() is deprecated

Your Java service is slow but CPU usage is low. What do you check first?

.

2. The Java application runs fine for hours and then suddenly slows down. What could be wrong?

The slowdown could be due to memory leaks, excessive garbage collection, or resource exhaustion, which can be identified using monitoring tools and fixed by optimizing memory and resource usage

3. You receive an OutOfMemoryError even though the heap appears sufficient. How is that possible?

4. Multiple threads update shared data, leading to inconsistent results. How do you fix it?

I would first identify the root cause using logs, monitoring, and profiling tools, and then apply appropriate fixes like synchronization, memory optimization, or resource management.

5. A deadlock occurs rarely and cannot be reproduced locally. How do you debug it?
6. GC pauses increased after a new release. What changes might have caused this?
7. Your JVM doesn't exit even after main() finishes. What's holding it?
8. A thread pool is configured correctly, but tasks are still getting delayed. Why?
9. High CPU usage occurs with very low traffic — where do you look?
10. A background job impacts API response time. How do you isolate it?
11. A HashMap grows in size, but lookups become slower. What went wrong?
12. Memory usage keeps increasing, but no obvious references exist. What do you suspect?

13. A task submitted to ExecutorService fails silently. What happened?
14. After scaling instances, overall performance becomes worse. Why?
15. An API is fast sometimes and slow at other times. What could cause this?
16. Logging was increased for debugging, and production went down. How?
17. Parallel streams were introduced, but latency increased. Why?
18. JVM tuning improved latency but reduced throughput. Explain the trade-off.
19. A retry mechanism caused system overload. What design mistake was made?
20. The application behaves differently between Java 8 and Java 17. Why

Explain your project architecture.
2. How is security handled in your project?
3. Explain JWT.
4. What annotations have you used in Spring Boot?
5. How do you implement Global Exception Handling?
6. Difference between @PathVariable and @RequestParam.
7. What is Dependency Injection?
8. Types of Dependency Injection in Spring.
9. What Java 8 features have you used in your project?
10. Explain Streams API.
11. Explain SOLID Principles.
12. Which Design Patterns have you used in your project?
13. Difference between Singleton Pattern and Factory Pattern.
14. What is Optional?
15. Find the 2nd non-repeating character in a String using Streams.
16. Find the 2nd highest salary from a list of employees.
17. How is deployment done in your project?

How does HashMap work internally?
Difference between ==, equals(), and hashCode()
Why String is immutable?
Fail-fast vs Fail-safe iterator
How garbage collection works in JVM
Explain memory areas in JVM (Heap, Stack, Metaspace)

- Multithreading & Concurrency (MOST IMPORTANT)

7. synchronized vs volatile vs AtomicInteger

9. ThreadPoolExecutor working
10. How you solved a deadlock issue
11. Race condition – real example
12. Producer-Consumer problem implementation

- Spring Boot (Every Interview)

13. How Spring Boot starts internally
14. @Transactional internal working
15. Filter vs Interceptor

16. Bean scopes & dependency injection
17. Exception handling in REST APIs
18. How you secured APIs (JWT / OAuth)

🔹 Database & JPA

19. Lazy vs Eager loading
20. N+1 query problem and fix
21. How indexing improves performance
22. How you debugged a slow query
23. First level vs Second level cache

🔹 Microservices & Production

24. How services communicate (Feign vs RestTemplate vs WebClient)
25. Circuit breaker & fallback
26. Idempotency in APIs

28. How you handled high CPU or memory leak
29. One production incident you solved

📌 Reality:

How can we make an existing HashMap thread-safe without using ConcurrentHashMap?
2. How do you ensure consistent data across multiple application instances? For example, if two JVM instances are running and a value is updated in App1 (JVM1), how will it be reflected in App2 (JVM2)?
3. Can we use @Transactional on protected or private methods?
4. What is the internal implementation of the @Transactional annotation?
5. Your application is deployed to production and experiencing high traffic. How would you handle it? What scaling strategies would you use?
6. How do you get alerts for 500 Internal Server Errors occurring every 5 minutes?
7. How do you handle partial failures in microservices architecture?
8. How would you log all methods annotated with @Transactional? (Provide pseudo code.)

11. How do you handle duplicate payments if a customer accidentally clicks the “Pay” button multiple times? How do you identify and inform the client about duplicates?
12. How do you achieve zero data loss in your application?

Mphasis

🟦 Core Java

1. What are virtual threads in Java 21 and how do they differ from traditional threads?

Virtual Threads are **lightweight threads** introduced as a standard feature in **Java 21** (Project Loom).

They allow you to create **millions of concurrent threads** with very low memory overhead.

Unlike platform threads, virtual threads are **managed by the JVM**, not directly by the operating system.

2. How does record improve DTO handling in Java?

Records simplify DTO creation by automatically generating constructors, accessors, equals(), hashCode(), and toString(). Since records are immutable by default, they are ideal for transferring data between application layers while reducing boilerplate code.

```
public record Employee(int id, String name) {}
```

3. Explain the difference between Optional.get(), orElse(), and orElseThrow().

Optional.get() returns the value but throws a NoSuchElementException if the Optional is empty, so it's generally avoided unless you're sure a value exists.

orElse() returns a default value when the Optional is empty.

orElseThrow() lets us throw a custom exception if no value is present, making it the preferred choice when the value is mandatory

4. How does ConcurrentHashMap achieve thread safety internally?

ConcurrentHashMap achieves thread safety by allowing multiple threads to read and write concurrently with minimal locking.

In **Java 8 and later**, it uses a combination of:

- CAS (Compare-And-Swap)
- Fine-grained locking (bucket-level locking)
- Volatile variables

Unlike Hashtable, which locks the entire map, ConcurrentHashMap locks only the bucket being modified, allowing better concurrency and performance.

5. What are switch expressions and how are they different from switch statements?

A **switch statement** is used to execute different blocks of code based on a condition, but it does not return a value. A **switch expression**, introduced in Java 14, can return a value directly, making the code more concise and reducing the need for break statements. It uses the -> syntax and helps avoid accidental fall-through.

6. Explain the Fork/Join framework and its advantages.

The Fork/Join Framework, introduced in Java 7, is designed for parallel processing using the divide-and-conquer approach. A large task is recursively divided into smaller subtasks using the fork() operation, executed in parallel by worker threads in a ForkJoinPool, and their results are combined using join(). It uses a work-stealing algorithm, allowing idle threads to steal tasks from busy threads, which improves CPU utilization and throughput

I use **ExecutorService** for general-purpose asynchronous tasks such as processing API requests, sending emails, or executing independent background jobs. I use the **Fork/Join Framework** when I have a large CPU-intensive task that can be recursively divided into smaller subtasks and executed in parallel, such as processing large arrays or implementing merge sort.

7. How does pattern matching for instanceof simplify Java code?

Combines type checking and casting into one step, reducing boilerplate.

9. What are the benefits of using streams and functional programming in Java?
10. How does Java handle memory management for unreachable objects?

java automatically manages memory using the Garbage Collector. When an object becomes unreachable, meaning no live references point to it,

🟦 Spring + Spring Boot

11. How does Spring Boot auto-configuration determine which beans to create?
12. What is the difference between @Component, @Service, and @Repository?
13. How do you configure global exception handling using @ControllerAdvice?
14. How do you secure REST APIs using Spring Security and JWT?
15. What are Spring Profiles and how do they help manage environments?
16. How does Spring Boot integrate with external configuration servers?
17. How do you implement scheduling in Spring Boot?
18. Explain how Spring Boot Actuator helps monitor applications.
19. How do you implement rate limiting and throttling in Spring Boot?
20. How do you manage dependency injection and circular dependencies in Spring?

🟦 Microservices

21. How do you implement service discovery in microservices architecture?
22. What is the difference between synchronous REST calls and asynchronous messaging?
23. How do you ensure fault tolerance using Circuit Breaker or Retry patterns?
24. How do you handle versioning of microservices APIs?

🟦 Coding Questions

25.Write a Java 8 stream code to group employees by department and get the highest salary per department.

1. What are virtual threads in Java 21 and how do they differ from traditional threads?
2. How does record improve DTO handling in Java?
3. Explain the difference between Optional.get(), orElse(), and orElseThrow().
4. How does ConcurrentHashMap achieve thread safety internally?
5. What are switch expressions and how are they different from switch statements?
6. Explain the Fork/Join framework and its advantages.
7. How does pattern matching for instanceof simplify Java code?
8. How do you implement immutability in Java classes?
9. What are the benefits of using streams and functional programming in Java?
10. How does Java handle memory management for unreachable objects?

🟦 Spring + Spring Boot

11. How does Spring Boot auto-configuration determine which beans to create?
12. What is the difference between @Component, @Service, and @Repository?
13. How do you configure global exception handling using @ControllerAdvice?
14. How do you secure REST APIs using Spring Security and JWT?
15. What are Spring Profiles and how do they help manage environments?
16. How does Spring Boot integrate with external configuration servers?
17. How do you implement scheduling in Spring Boot?
18. Explain how Spring Boot Actuator helps monitor applications.
19. How do you implement rate limiting and throttling in Spring Boot?
20. How do you manage dependency injection and circular dependencies in Spring?

🟦 Microservices

21. How do you implement service discovery in microservices architecture?
22. What is the difference between synchronous REST calls and asynchronous messaging?
23. How do you ensure fault tolerance using Circuit Breaker or Retry patterns?
24. How do you handle versioning of microservices APIs?

🟦 Coding Questions

25. Write a Java function to reverse words in a sentence in place.
26. Implement a method to find the kth largest element in an array.
27. Write a program to detect cycles in a directed graph.

🟧 Others

29. What is the difference between StatefulSet and Deployment in Kubernetes?
30.
31. How do you implement idempotent REST APIs?
32. How do you configure persistent storage in Docker and Kubernetes?
33. What is the difference between Docker images and containers?
34. How do you implement blue-green or canary deployments in Kubernetes?

Core Java

1. What is the internal working of ConcurrentHashMap in Java 8?
2. Explain how JVM handles class loading and garbage collection.
3. What is the difference between volatile, synchronized, and Lock?
4. How does the ForkJoinPool work internally?
5. Explain immutability and how to create a truly immutable class.

Spring & Spring Boot

6. What is the difference between @Component, @Service, and @Repository?
7. How does Spring manage bean lifecycle and dependency injection internally?
8. Explain how @Transactional works under the hood.
9. What is the difference between Spring MVC and Spring WebFlux?
10. How does Spring Boot auto-configuration work?

Microservices

11. Explain API Gateway pattern & how routing + filtering works.
12. How do you handle inter-service communication failures? (Retry, Circuit Breaker, Fallback)
13. What is service discovery? How does Eureka/Consul work internally?
14. Explain eventual consistency in microservices with an example.
15. How do you design idempotent APIs?

Database & Performance

16. How do you optimize slow SQL queries?
17. Explain different isolation levels & the problems they prevent.
18. What is connection pooling & how HikariCP optimizes performance?
19. How does caching work in distributed systems?
20. How do you design high-performance REST APIs?

System Design

21. Design a URL Shortener / Payment Service / Notification Service.
22. Explain load balancing strategies (Round-Robin, Least Connection, Weighted).
23. How do you scale a microservices-based application?
24. What logging & monitoring stack will you choose and why?
25. How do you secure microservices (OAuth2, JWT, API Tokens)?

SQL

- SQL Query to find the second highest salary.

```
SELECT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET n-1;
```

Write a SQL query to find duplicate records in a table.

22) Difference between SQL and No

- Given an employee table with columns: empId, name, salary, department
→ Write a query to find the average salary of each department

Select dept*id max(salary) as maxi from employee group by dept*id.
- Difference between INNER JOIN and OUTER JOIN.

Question 2: SQL Query to find Max Salary from each department.

```
SELECT department_id ,Max(salary) FROM employees GROUP BY department_id

ELECT department_id,COUNT(*) AS total,AVG(salary) AS average,MAX(salary) AS MAXi FROM employees GROUP BY department_id;
```

Select avg and count of emp each dept

Select * from employee Where salary>(Select avg(salary) from employee)

Duplicates record:

Select emp*id ,count()* ***from employee group by emp*name having count(***)>1

Show emp names from each dept with max salary

```
SELECT e.* FROM employees e JOIN(SELECT department_id,Max(salary) as max_sal FROM employees GROUP BY department_id) d on e.department_id=d.department_id AND e.salary=d.max_sal;
```

View

A **View** in SQL is a **virtual table** created from the result of a SQL query. It does **not store data itself** . Instead, it stores the SQL query, and whenever you query the view, the database executes that query and returns the result.

Index:

Indexing is a technique used by a database to **speed up data retrieval**. An index works much like the index in a book—it helps the database find rows quickly without scanning the entire table.

Normalization :

Normalization is the process of organizing data in a database to **reduce redundancy (duplicate data)** and **improve data integrity**.

First Normal Form (1NF)

Rule:

- Each column should contain only atomic (single) values.
- No repeating groups or arrays.

2NF removes **partial dependencies**. If a table has a composite primary key, every non-key attribute must depend on the entire key. For example, if StudentName depends only on StudentID and CourseName depends only on CourseID, we split the data into separate Student, Course, and Enrollment tables.

3NF removes **transitive dependencies**. Every non-key attribute should depend only on the primary key, not on another non-key attribute. For example, DepartmentName should be stored in a separate Department table instead of the Employee table because it depends on DepartmentID, not directly on EmployeeID.

A candidate key is any column or combination of columns that can uniquely identify each row in a table

"A candidate key is any column or combination of columns that can uniquely identify each row in a table. A table can have multiple candidate keys. A primary key is the candidate key selected to uniquely identify records in the table. For example, in an Employee table, EmployeeID, Email, and Aadhaar may all be unique, making them candidate keys. If we choose EmployeeID as the primary key, the remaining candidate keys become alternate keys."

Where vs having

WHERE

- Filters rows **before** grouping.
- Cannot use aggregate functions.

HAVING

- Filters groups **after** GROUP BY.
- Can use aggregate functions.

INNER JOIN

Returns matching records.

Find Duplicate Records

```
SELECT email ,count(*) FROM employees GROUP BY email HAVING COUNT(*)>1;
```

Find Employees Without Department

```
SELECT * FROM employees e LEFT JOIN departments d on e.department_id=d.department_id WHERE d.department_id is NULL;
```

COUNT(*) vs COUNT(column)

COUNT(*) -> Counts all rows

COUNT(email) -> Ignores NULL values

Find Top 3 Highest Salaries

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 3;
```

Follow-up Question

Can a table have multiple primary keys?

❌ No. A table can have **only one primary key** (though it can consist of multiple columns, called a composite primary key).

Can a table have multiple candidate keys?

✅ Yes. A table can have **multiple candidate keys**, but only one of them is selected as the primary key.

BCNF says:

If a column determines another column, then that column must be a candidate key.

In other words, any column or combination of columns that determines another column must uniquely identify a row

Kafka

🔹 If Kafka is down, what will happen?

If Kafka is unavailable, the producer won't be able to publish messages. Depending on the business requirement, I can retry with exponential backoff, temporarily store messages in an outbox table (Outbox Pattern), or return an appropriate error to the client. Using the Outbox Pattern ensures that messages are not lost and are published once Kafka becomes available again.

OFFSET:

Offset is a unique sequential number assigned to every message **within a Kafka partition**. A consumer uses the offset to keep track of which messages it has already processed. When the consumer reads messages, it commits the latest processed offset. If the consumer restarts, it resumes reading from the last committed offset instead of reading all messages again.

25. What problems did Kafka solve in your project?

Kafka solved the problem of tight coupling and synchronous communication in our system. It enabled asynchronous event-driven communication between microservices, allowing producers and consumers to work independently. This improved performance, scalability, and fault tolerance. Kafka also helped buffer traffic spikes, handle large volumes of messages, and ensure reliable message delivery even when downstream services were temporarily unavailable.

26. How do you avoid duplicate Kafka message processing?

We avoid duplicate Kafka message processing by implementing idempotent consumers. Each message contains a unique identifier, and before processing we check whether it has already been processed. We commit offsets only after successful processing and may also use database unique constraints or Kafka's exactly-once semantics to handle duplicates safely.

Example:
Check transaction ID before processing.

27. What happens if Kafka consumer crashes while processing data?

If a Kafka consumer crashes while processing a message, Kafka detects the failure because the consumer stops sending heartbeats. The consumer is removed from the consumer group, and a rebalance occurs. The partitions assigned to the failed consumer are reassigned to another consumer. Processing resumes from the last committed offset. If the consumer crashes before committing the offset, the message will be consumed again when the consumer restarts, which can lead to duplicate processing. If the offset was already committed, Kafka assumes the message was processed successfully and will not re-deliver it.

If offset not committed:

Kafka reprocesses message

If committed early:

Message loss possible

Best practice:

Commit offset after successful processing

- Explain Kafka architecture.

Kafka follows a publish-subscribe architecture. Producers publish messages to topics, which are divided into partitions for scalability and parallel processing. These partitions are stored on Kafka brokers. Consumers read messages from topics, and consumers within the same consumer group share the work by consuming different partitions. This architecture provides high throughput, fault tolerance, and scalability.

- What are Topics, Partitions, and Consumer Groups in Kafka?

A topic is a logical channel used to categorize messages. Each topic is divided into one or more partitions, allowing Kafka to process data in parallel while maintaining message order within each partition.

A consumer group is a collection of consumers that work together to process messages from a topic, with each partition assigned to only one consumer in the group.

What is Dead Letter Queue (DLQ)?

A Dead Letter Queue is a separate Kafka topic where messages that fail processing after the configured number of retries are sent. This prevents problematic messages from blocking the processing of other messages and allows them to be analyzed or reprocessed later.

28. How does Kafka handle partition rebalancing when a consumer joins or leaves?

9. If your Kafka-based application is deployed with multiple consumer instances, how do you ensure that only one consumer instance processes messages from a topic? Which configuration/property would you use?
10. How do you handle failures during Kafka message consumption in microservices?

27. Handling duplicate Kafka messages

- If Kafka is down, what will happen?

11. Kafka consumer lag keeps increasing. What would you investigate?

Deployement

Kubectl cmd:kubectl scale deployment appname –replicas=4

Kubectl apply –f yaml*file*name

Deployment is used to create and manage application Pods. It specifies details like the Docker image, the number of replicas, and how updates should be performed.

A Service exposes those Pods through a stable endpoint and load-balances traffic across them.

Spring Boot Deployment Steps (Docker + Kubernetes)

Step 1: Develop the application

- Write the Spring Boot code.
- Test it locally.

Step 2: Build the application

Use Maven to generate the JAR file.

mvn clean package

Output:

target/app.jar

Step 3: Create a Docker image

Write a Dockerfile and build the image.

docker build -t order-service:1.0 .

Step 4: Push the image to a Docker Registry

Push the image so Kubernetes can download it.

docker push order-service:1.0

Step 5: Create Kubernetes YAML files

Create the configuration files required by Kubernetes.

- **Deployment.yaml** → Defines the Docker image, number of Pods (replicas), and update strategy.
- **Service.yaml** → Exposes the Pods through a stable endpoint and load-balances traffic.
- **ConfigMap.yaml** *(optional)* → Stores configuration such as URLs or environment variables.
- **Secret.yaml** *(optional)* → Stores sensitive data like passwords or API keys.
- **Ingress.yaml** *(optional)* → Allows external users to access the application.

Step 6: Deploy to Kubernetes

Apply the YAML files to the Kubernetes cluster.

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

Kubernetes creates the required Pods automatically.

Step 7: Verify the deployment

kubectl get pods
kubectl get services
kubectl logs <pod-name>

Check that the Pods are running and the application is healthy.

Step 8: Scale the application (if needed)

kubectl scale deployment order-service --replicas=5

Kubernetes starts additional Pods.

Step 9: Deploy a new version (Rolling Update)

1. Build a new Docker image.
2. Push it to the registry.
3. Update the Kubernetes deployment.

docker build -t order-service:2.0 .
docker push order-service:2.0

kubectl set image deployment/order-service order-service=order-service:2.0

Kubernetes gradually replaces the old Pods with the new ones without downtime.

End-to-End Flow

Write Code

↓

mvn clean package

↓

Create Docker Image

↓

Push Image to Docker Registry

↓

Create Kubernetes YAML Files

↓

kubectl apply

↓

Kubernetes Creates Pods

↓

Service Exposes the Application

↓

Verify Deployment

↓

Scale / Rolling Updates